

Análise Exploratória

Aqui iniciamos nossa análise exploratória da base de dados. Como o arquivo original possuía 6Gb de tamanho e com a limpeza de dados chegamos a ter um volume de 2Gb.

1. Importação dos dados

Funções auxiliares e importação das bibliotecas

```
In [252... import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import textwrap
import json

def quebrar_texto(texto, largura=15):
    return "\n".join(textwrap.wrap(texto, largura))

def carregar_paleta(paleta=False):
    if not paleta:
        return None

    with open(paleta, "r", encoding="utf-8") as arquivo:
        dados = json.load(arquivo)

    return dados["palette"]

def plot_media(df, coluna, group_col, titulo, paleta=False):

    cores = carregar_paleta(paleta)

    df_plot = (
        df
```

```
        .groupby(group_col)[coluna]
        .mean()
        .sort_values(ascending=False)
    )

plt.figure(figsize=(6, 3))

if cores:
    ax = df_plot.plot(
        kind="bar",
        color=cores["primary_blue"]
    )
else:
    ax = df_plot.plot(kind="bar")

plt.title(titulo)
plt.xlabel(group_col)
plt.ylabel(f"Média de {coluna}")
plt.xticks(rotation=45)

for i, v in enumerate(df_plot):
    ax.text(i, v, f"{v:.2f}", ha="center", va="bottom")

plt.tight_layout()
plt.show()

def plot_barra_empilhada(df, eixo_x, coluna_stack, top_n=None, sort_total=True, ascending=False, figsize=(10, 5), titulo=None,
                        cores = carregar_paleta(paleta)):

    df_plot = (
        df
        .groupby([eixo_x, coluna_stack])
        .size()
        .unstack(fill_value=0)
    )

    df_plot["total"] = df_plot.sum(axis=1)

    if sort_total:
```

```
df_plot = df_plot.sort_values(  
    by="total",  
    ascending=ascending  
)  
  
if top_n:  
    df_plot = df_plot.head(top_n)  
  
if largura_texto:  
  
    df_plot.index = df_plot.index.map(  
        lambda x: quebrar_texto(x, largura_texto)  
    )  
  
cores_barras = None  
  
if cores:  
  
    lista_cores = [  
        cores["primary_blue"],  
        cores["primary_green"],  
        cores["gold"],  
        cores["soft_green"],  
        cores["light_gray"]  
    ]  
  
    qtd_colunas = len(  
        df_plot.drop(columns="total").columns  
    )  
  
    cores_barras = (  
        lista_cores * qtd_colunas  
   )[:qtd_colunas]  
  
ax = df_plot.drop(columns="total").plot(  
    kind="bar",  
    stacked=True,  
    figsize=figsize,  
    color=cores_barras  
)
```

```
plt.title(
    titulo if titulo
    else f"Distribuição de {coluna_stack} por {eixo_x}"
)

plt.xlabel(
    xlabel if xlabel
    else eixo_x
)

plt.ylabel(ylabel)

plt.xticks(rotation=rotacao)

if mostrar_total:
    for i, total in enumerate(df_plot["total"]):
        ax.text(
            i,
            total,
            f"{int(total):,}",
            ha="center",
            va="bottom",
            fontsize=9,
            fontweight="bold"
        )

plt.legend(
    title=(
        legenda_titulo
        if legenda_titulo
        else coluna_stack
    )
)

plt.tight_layout()
plt.show()

def plot_barra(df, eixo_x, coluna_valor=None, aggfunc="count", top_n=None, sort=True, ascending=False, figsize=(6, 4), titulo=
```

```
cores = carregar_paleta(paleta)

# AGRUPAMENTO
if coluna_valor:

    if aggfunc == "count":
        df_plot = (
            df
            .groupby(eixo_x)[coluna_valor]
            .count()
        )

    elif aggfunc == "sum":
        df_plot = (
            df
            .groupby(eixo_x)[coluna_valor]
            .sum()
        )

    elif aggfunc == "mean":
        df_plot = (
            df
            .groupby(eixo_x)[coluna_valor]
            .mean()
        )

    else:
        raise ValueError(
            "aggfunc deve ser: count, sum ou mean"
        )

else:
    df_plot = df[eixo_x].value_counts()

# ORDENAÇÃO
if sort:
    df_plot = df_plot.sort_values(
        ascending=ascending
    )

# TOP N
```

```
if top_n:
    df_plot = df_plot.head(top_n)

# QUEBRA TEXTO
if largura_texto:
    df_plot.index = df_plot.index.map(
        lambda x: quebrar_texto(x, largura_texto)
    )

# PLOT
plt.figure(figsize=figsize)

ax = df_plot.plot(
    kind="bar",
    color=cores["primary_blue"] if cores else None
)

plt.title(
    titulo if titulo
    else f"Distribuição por {eixo_x}"
)

plt.xlabel(
    xlabel if xlabel
    else eixo_x
)

plt.ylabel(ylabel)

plt.xticks(rotation=rotacao)

# RÓTULOS
for i, v in enumerate(df_plot):
    ax.text(
        i,
        v,
        formato_rotulo.format(v),
        ha="center",
        va="bottom"
    )
```

```
plt.tight_layout()
plt.show()

def plot_barra_falencia(df, eixo_x, coluna_status="status", status_ativo="ATIVO", status_faliu="FALIU", minimo_registros=2000,

cores = carregar_paleta(paleta)

df_plot = (
    df
    .groupby([eixo_x, coluna_status])
    .size()
    .unstack(fill_value=0)
)

df_plot = df_plot.reindex(
    columns=[status_ativo, status_faliu],
    fill_value=0
)

df_plot["total"] = df_plot[status_ativo] + df_plot[status_faliu]
df_plot["taxa_falencia"] = df_plot[status_faliu] / df_plot["total"]

df_plot = df_plot[df_plot["total"] > minimo_registros]

if pareto:

    df_plot = df_plot.sort_values(
        by="taxa_falencia",
        ascending=False
    )

    df_plot["pareto"] = (
        df_plot["total"].cumsum()
        / df_plot["total"].sum()
    )

    df_plot = df_plot[df_plot["pareto"] <= 0.80]
    df_plot = df_plot.head(top_n)

coluna_barra = "taxa_falencia"
```

```
        ylabel = "Taxa de Falência"

    else:

        df_plot = (
            df_plot
            .sort_values(by="total", ascending=False)
            .head(top_n)
        )

        coluna_barra = "total"
        ylabel = "Quantidade"

    df_plot.index = df_plot.index.map(
        lambda x: quebrar_texto(x, largura_texto)
    )

    fig, ax1 = plt.subplots(figsize=figsize)

    ax1.bar(
        df_plot.index,
        df_plot[coluna_barra],
        color=cores["primary_blue"] if cores else None
    )

    ax1.set_title(
        titulo if titulo
        else f"Quantidade e Taxa de Falência por {eixo_x} - TOP {top_n}"
    )

    ax1.set_xlabel(eixo_x)
    ax1.set_ylabel(ylabel)

    plt.xticks(rotation=45, ha="right")

    for i, (total, taxa) in enumerate(
        zip(df_plot["total"], df_plot["taxa_falencia"])
    ):

        label = f"{int(total):,} | {taxa:.0%}"
```



```
ax1.text(
    i,
    taxa if pareto else total,
    label,
    ha="center",
    va="bottom",
    fontsize=9
)

if pareto:

    ax2 = ax1.twinx()

    ax2.plot(
        df_plot.index,
        df_plot["pareto"],
        marker="o",
        color=cores["gold"] if cores else None
    )

    ax2.set_ylabel("Pareto Acumulado")
    ax2.set_ylim(0, 1.05)

    ax2.axhline(
        0.8,
        linestyle="--",
        color=cores["primary_green"] if cores else None
    )

    for i, valor in enumerate(df_plot["pareto"]):

        ax2.text(
            i,
            valor,
            f"{valor:.0%}",
            ha="center",
            va="bottom",
            fontsize=8
        )

plt.tight_layout()
```

```
plt.show()

df_plot = df_plot.reset_index()

return df_plot
```

```
PALETA = r"C:\Users\leona\OneDrive\Documentos\MBA Eng Dados\Projeto Final de curso\eng-tech\paleta.json"
```

```
In [253... df = pd.read_csv(
    r"C:\Users\leona\OneDrive\Documentos\MBA Eng Dados\Projeto Final de curso\base de dados\camada_silver_v2.csv",
    sep=";",
    encoding="latin-1"
)
```

C:\Users\leona\AppData\Local\Temp\ipykernel_16796\1031811550.py:1: DtypeWarning: Columns (14,15,34,35) have mixed types. Specify dtype option on import or set low_memory=False.

```
df = pd.read_csv(
```

```
In [254... cnae = pd.read_csv(
    r"C:\Users\leona\OneDrive\Documentos\MBA Eng Dados\Projeto Final de curso\base de dados\cnae.csv",
    sep=";",
    encoding="utf-8"
)
```

```
In [255... municipio = pd.read_csv(
    r"C:\Users\leona\OneDrive\Documentos\MBA Eng Dados\Projeto Final de curso\base de dados\municipio.csv",
    sep=";",
    encoding="utf-8"
)
```

2. Iniciando análise do dataframe

2.1 Analisando características do dataframe e ajuste de formatos

- Target: Status

Esse é nome da coluna que diz se a empresa faliu ou não. Elas se baseia na coluna "Situação Cadastral", se 8 então faliu senão ativo.

```
In [256... mapeamento = {2: "ATIVO", 8: "FALIU"}  
df["status"] = df["situacao_cadastral"].map(mapeamento)
```

- Definindo o porte das empresas

Se o valor de porte_empresas for 5 então grande porte, se 1 micro empresa e se for 3 empresa de pequeno porte.

```
In [257... mapeamento = {1: "Micro", 3: "Pequeno", 5: "Demais"}  
df["porte_da_empresa"] = df["porte_empresa"].map(mapeamento)
```

- Acrescentando descrição dos código CNAE (Classificação Nacional Atividades Econômicas) principal e secundário, código da prestação de serviço, para auxiliar na análise.

```
In [258... # transformar cnae em dicionário  
cnae_dict = cnae.set_index("codigo")["descricao"].to_dict()  
  
df["descricao_cnae"] = df["cnae_fiscal_principal"].map(cnae_dict)  
  
df["descricao_cnae_sec"] = df["cnae_fiscal_secundaria"].map(cnae_dict)
```

- Identificando cada município pelo código

```
In [259... # transformar cnae em dicionário  
municipio_dict = municipio.set_index("codigo")["descricao"].to_dict()  
  
df["municipio_ident"] = df["municipio"].map(municipio_dict)
```

```
In [260... df['municipio_ident'].head(5)
```

```
Out[260... 0    JABOATAO DOS GUARARAPES
1    SAO JOSE DO RIO PARDO
2              NAVIRAI
3    PONTA GROSSA
4    SAO PAULO
Name: municipio_ident, dtype: object
```

- Informações Gerais do Dataframe

```
In [261... colunas = df.columns.tolist()

print(f"Tamanho do DataFrame amostrado (linhas, colunas): {df.shape}\n")
print("Colunas:")

for i in range(0, len(colunas), 5):
    linha = " | ".join(colunas[i:i + 5])
    print(linha)
```

Tamanho do DataFrame amostrado (linhas, colunas): (9136524, 44)

Colunas:

```
cnpj_basico | cnpj_ordem | cnpj_dv | identificador_mf | nome_fantasia
situacao_cadastral | data_situacao_cadastral | motivo_situacao_cad | pais | data_inicio_atividade
cnae_fiscal_principal | cnae_fiscal_secundaria | uf | municipio | situacao_especial
data_situacao_especial | empresa_mais_5_anos | tempo_vida | razao_social | natureza_juridica
qualif_responsavel | capital_social | porte_empresa | ente_federativo_resp | identificador_socio
qualif_socio | data_entrada_sociedade | pais_socio | qualif_representante | faixa_etaria
opcao_simples | data_opcao_simples | data_exclusao_simples | opcao_mei | data_opcao_mei
data_exclusao_mei | tempo_exclusao_mei | tempo_op_simples | tempo_excl_op_simples | status
porte_da_empresa | descricao_cnae | descricao_cnae_sec | municipio_ident
```

```
In [262... print(display(df.head(3)))
```

	cnpj_basico	cnpj_ordem	cnpj_dv	identificador_mf	nome_fantasia	situacao_cadastral	data_situacao_cadastral	motivo_situacao_cad	pais
0	5597167	1	3	1	BR REBOQUE	2	2025-01-05	0	
1	6270091	2	42	2	NaN	8	2013-08-15	1	
2	19063605	1	50	1	MEGA BIJU PRECO UNICO 1,99	8	2018-05-29	1	

3 rows × 44 columns

None

- Informações dos tipos de cada coluna, no caso temos muitas colunas classificadas como tipo "Object", por isso será necessário corrigir os tipos para carregar na IA corretamente.

In [263... `df.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9136524 entries, 0 to 9136523
Data columns (total 44 columns):
#   Column                                Dtype
---  -
0   cnpj_basico                          int64
1   cnpj_ordem                          int64
2   cnpj_dv                             int64
3   identificador_mf                    int64
4   nome_fantasia                       object
5   situacao_cadastral                  int64
6   data_situacao_cadastral             object
7   motivo_situacao_cad                 int64
8   pais                                object
9   data_inicio_atividade              object
10  cnae_fiscal_principal                int64
11  cnae_fiscal_secundaria              object
12  uf                                   object
13  municipio                           int64
14  situacao_especial                   object
15  data_situacao_especial              object
16  empresa_mais_5_anos                 bool
17  tempo_vida                          float64
18  razao_social                        object
19  natureza_juridica                   int64
20  qualif_responsavel                  int64
21  capital_social                      float64
22  porte_empresa                       int64
23  ente_federativo_resp               float64
24  identificador_socio                 int64
25  qualif_socio                        int64
26  data_entrada_sociedade              object
27  pais_socio                          object
28  qualif_representante                int64
29  faixa_etaria                        int64
30  opcao_simples                       object
31  data_opcao_simples                  object
32  data_exclusao_simples               object
33  opcao_mei                           object
34  data_opcao_mei                      object
35  data_exclusao_mei                   object
```

```
36 tempo_exclusao_mei      float64
37 tempo_op_simples        float64
38 tempo_excl_op_simples    float64
39 status                   object
40 porte_da_empresa         object
41 descricao_cnae           object
42 descricao_cnae_sec       object
43 municipio_ident          object
dtypes: bool(1), float64(6), int64(15), object(22)
memory usage: 2.9+ GB
```

- Garantindo que estamos analisando apenas empresas e não MEI.

```
In [264...] df = df[df["opcao_mei"] == "N"].copy()
```

- Para a análise vamos estudar as empresas de pequeno porte ou menos, pois são elas que estão sujeitas a maior probabilidade de falência.

```
In [265...] df = df[df["porte_empresa"] != 5].copy()
```

- Ajustando valores numéricos relevantes

```
In [266...] colunas_data = [
    "capital_social"
]

for col in colunas_data:
    if col in df.columns:
        df[col] = pd.to_numeric(
            df[col],
            errors="coerce"
        )
```

- Ajustando datas

```
In [267... colunas_data = [  
    "data_inicio_atividade",  
    "data_situacao_cadastral",  
    "data_situacao_especial",  
    "data_entrada_sociedade",  
    "data_opcao_simples",  
    "data_exclusao_simples",  
    "data_opcao_mei",  
    "data_exclusao_mei"  
]  
  
for col in colunas_data:  
    if col in df.columns:  
        df[col] = pd.to_datetime(  
            df[col],  
            errors="coerce"  
        )  
  
# Colunas auxiliares a partir das datas (ano), será dividido por 365.25 para obter o tempo em anos, considerando anos bissextos  
df["tempo_exclusao_mei"] = (df["data_exclusao_mei"] - df["data_inicio_atividade"]).dt.days.astype(float) / 365.25  
df["tempo_op_simples"] = (df["data_opcao_simples"] - df["data_inicio_atividade"]).dt.days.astype(float) / 365.25  
df["tempo_excl_op_simples"] = (df["data_exclusao_simples"] - df["data_opcao_simples"]).dt.days.astype(float) / 365.25  
  
# Tratamento de zeros que não podem existir  
cols_tempo = [  
    "tempo_exclusao_mei",  
    "tempo_op_simples",  
    "tempo_excl_op_simples"  
]  
  
for col in cols_tempo:  
    df.loc[df[col] < 0, col] = None
```

- Identificando situação jurídica das empresas

```
In [268... map_sit_juridica = {  
    "3999": "Associação Privada",  
    "2046": "Sociedade Anônima Aberta",
```



```

"2054": "Sociedade Anônima Fechada",
"2062": "Sociedade Empresária Limitada",
"2070": "Sociedade Empresária em Nome Coletivo",
"2089": "Sociedade Empresária em Comandita Simples",
"2100": "Sociedade Mercantil de Capital e Indústria",
"2135": "Empresário (Individual)",
"2232": "Sociedade Simples Pura",
"2240": "Sociedade Simples Limitada",
"2259": "Sociedade Simples em Nome Coletivo",
"2267": "Sociedade Simples em Comandita Simples",
"2305": "Empresa Individual de Responsabilidade Limitada (de Natureza Empresária)",
"2313": "Empresa Individual de Responsabilidade Limitada (de Natureza Simples)",
"2321": "Sociedade Unipessoal de Advocacia",
"2330": "Cooperativas de Consumo",
"2348": "Empresa Simples de Inovação"
}

df["natureza_juridica"] = df["natureza_juridica"].astype(str)

df["sit_juridica"] = df["natureza_juridica"].map(map_sit_juridica)

```

- Inclusão de coluna com faixa de valores para análise comparativa do capital inicial, uma observação importante que esse valor pode ser um fator complementar e não definidor ou mais importante, pois esse valor pode não representa o valor da empresa para mercado ou o faturamento dela.

```

In [269... # Criando faixas de capital social

bins = [0, 1000, 10000, 50000, 100000, 500000, 1000000, 10000000, float("inf")]

labels = [
    "0-1k",
    "1k-10k",
    "10k-50k",
    "50k-100k",
    "100k-500k",
    "500k-1M",
    "1M-10M",
    "10M+"

```

```
]

df["faixa_capital"] = pd.cut(
    df["capital_social"],
    bins=bins,
    labels=labels
)
```

- Inclusão de coluna definindo a qual região do país pertence a respectiva unidade federativa (UF)

In [270...

```
regioes = {

    "AC": "Norte",
    "AP": "Norte",
    "AM": "Norte",
    "PA": "Norte",
    "RO": "Norte",
    "RR": "Norte",
    "TO": "Norte",

    "AL": "Nordeste",
    "BA": "Nordeste",
    "CE": "Nordeste",
    "MA": "Nordeste",
    "PB": "Nordeste",
    "PE": "Nordeste",
    "PI": "Nordeste",
    "RN": "Nordeste",
    "SE": "Nordeste",

    "DF": "Centro-Oeste",
    "GO": "Centro-Oeste",
    "MT": "Centro-Oeste",
    "MS": "Centro-Oeste",

    "ES": "Sudeste",
    "MG": "Sudeste",
    "RJ": "Sudeste",
    "SP": "Sudeste",
```

```

    "PR": "Sul",
    "RS": "Sul",
    "SC": "Sul",

    # Exterior
    "EX": "Exterior"
}

df["regiao"] = df["uf"].map(regioes)

```

- Adicionando coluna com o tempo de funcionamento da empresa de acordo com o critério:
 - se status = FALIU, então o tempo de vida será a data_situacao_cadastral - data_inicio_atividade, tempo em anos
 - se status <> FALIU, então data de hoje - data_inicio_atividade, tempo em anos

```

In [271... hoje = pd.Timestamp("today")

df["tempo_funcionamento"] = np.where(
    df["status"] == "FALIU",
    # Empresas encerradas
    ((df["data_situacao_cadastral"] - df["data_inicio_atividade"])).dt.days / 365.25,

    # Empresas ativas
    (hoje - df["data_inicio_atividade"])).dt.days / 365.25
)

```

- Range início de empresas no estudo

```

In [272... print(f"Data mais anntiga para 'data_inicio_atividade': {df['data_inicio_atividade'].min()}")
print(f"Data mais recente para 'data_inicio_atividade': {df['data_inicio_atividade'].max()}")

```

Data mais anntiga para 'data_inicio_atividade': 1920-11-11 00:00:00

Data mais recente para 'data_inicio_atividade': 2025-11-08 00:00:00

Temos empresas antigas misturadas com empresas abertas recentemente, tendo um bom range de informação para o estudo.

2.2 Explorando datasets

Vamos copiar o dataframe para um novo para garantir que os dados originais não sejam corrompidos e evitar a necessidade de carregar eles novamente em caso de erro.

```
In [273... df_analise = df.copy()
```

Perguntas iniciais para conhecermos o dataframe:

- Qual é a taxa de "falência"?
- Qual é a taxa de "falência" por porte?
- Existe diferença por região (UF)?
- Empresas mais jovens falem mais?
- Capital social influencia a sobrevivência?

Qual é a taxa de "falência"?

```
In [274... print(f"Taxa de falência: {len(df_analise[df_analise['status'] == 'FALIU']) / len(df_analise) * 100:2.2f}%")
```

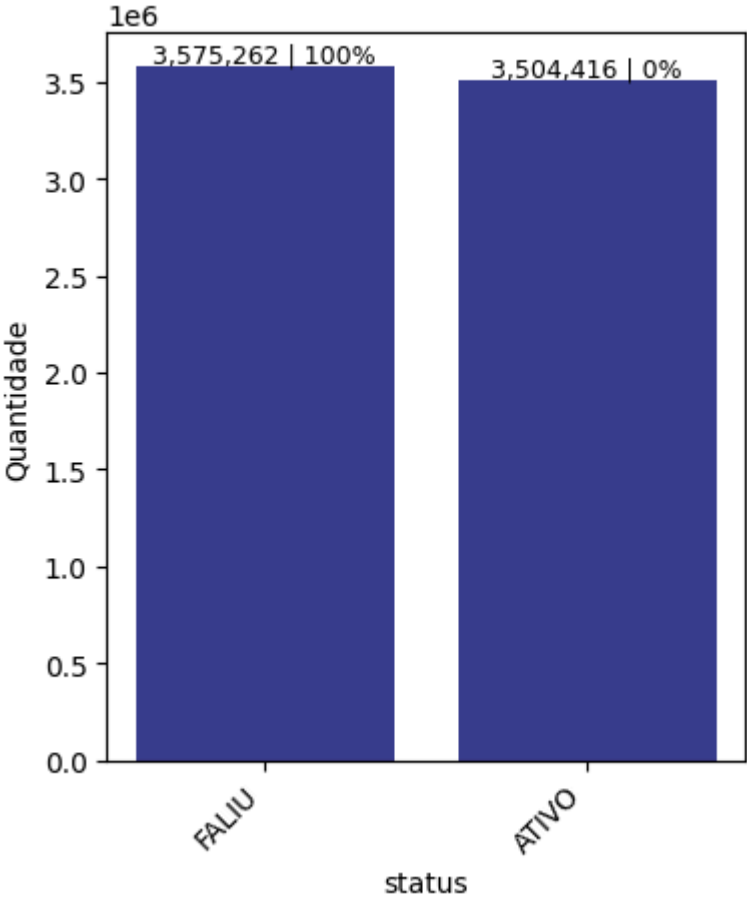
Taxa de falência: 50.50%

Mais da metade das empresas no dataset tiveram um registro de falência para o mês 10/25.

```
In [275... returned_df = plot_barra_falencia(  
    df_analise,  
    eixo_x="status",  
    coluna_status="status",  
    status_ativo="ATIVO",  
    status_faliu="FALIU",  
    minimo_registros=0,  
    top_n=2,  
    pareto=False,  
    largura_texto=20,  
    figsize=(4, 5),  
    titulo=None,  
    paleta=PALETA
```

```
)  
returned_df
```

Quantidade e Taxa de Falência por status - TOP 2

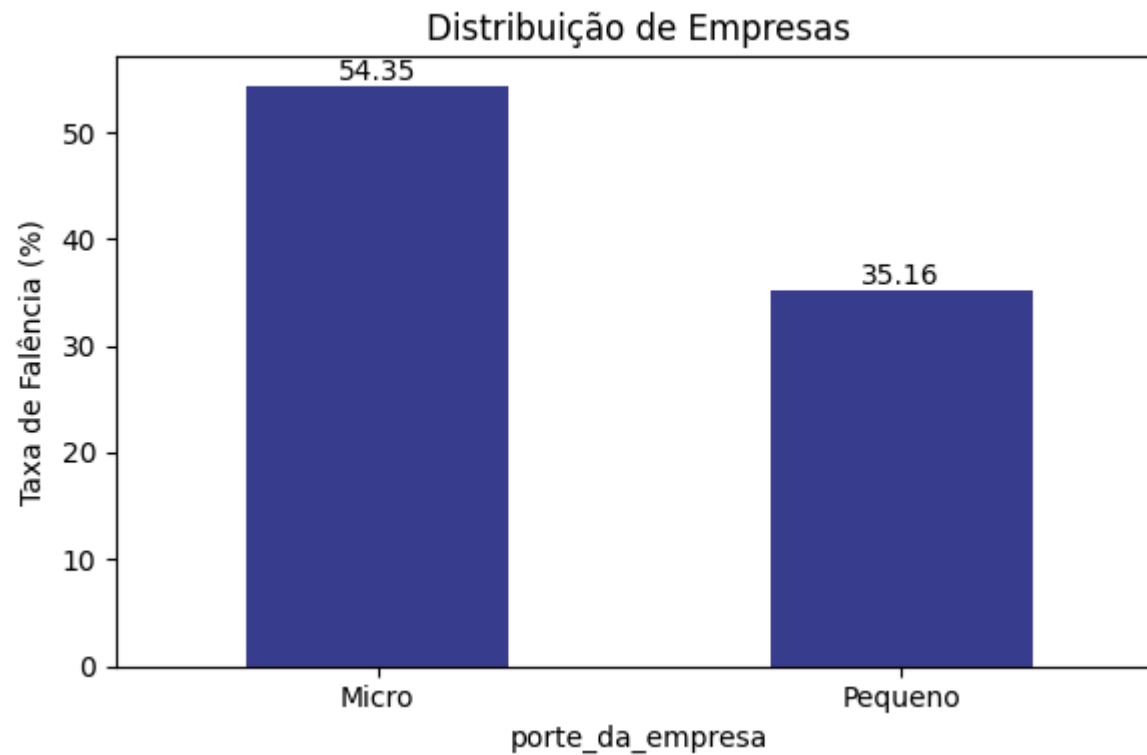


Out[275...

	status	status	ATIVO	FALIU	total	taxa_falencia
0		FALIU	0	3575262	3575262	1.0
1		ATIVO	3504416	0	3504416	0.0

Qual é a taxa de "falência" por porte?

```
In [276... df_porte = (  
    df_analise  
    .groupby(["porte_da_empresa", "status"])  
    .size()  
    .unstack(fill_value=0)  
)  
  
# taxa  
df_porte["taxa_falencia"] = (  
    round((df_porte["FALIU"] / (df_porte["FALIU"] + df_porte["ATIVO"]))*100, 2)  
)  
  
df_porte = df_porte.reset_index()  
  
df_porte = df_porte.drop(columns=["ATIVO", "FALIU"])  
  
df_porte = df_porte.reset_index()  
  
plot_barra(  
    df=df_porte,  
    eixo_x="porte_da_empresa",  
    coluna_valor="taxa_falencia",  
    aggfunc="mean",  
    ylabel="Taxa de Falência (%)",  
    titulo="Distribuição de Empresas",  
    paleta=PALETA  
)
```



Micro 54% Pequeno 35%

Existe diferença por região (UF)?

Verificando primeiro a taxa de falência para cada região.

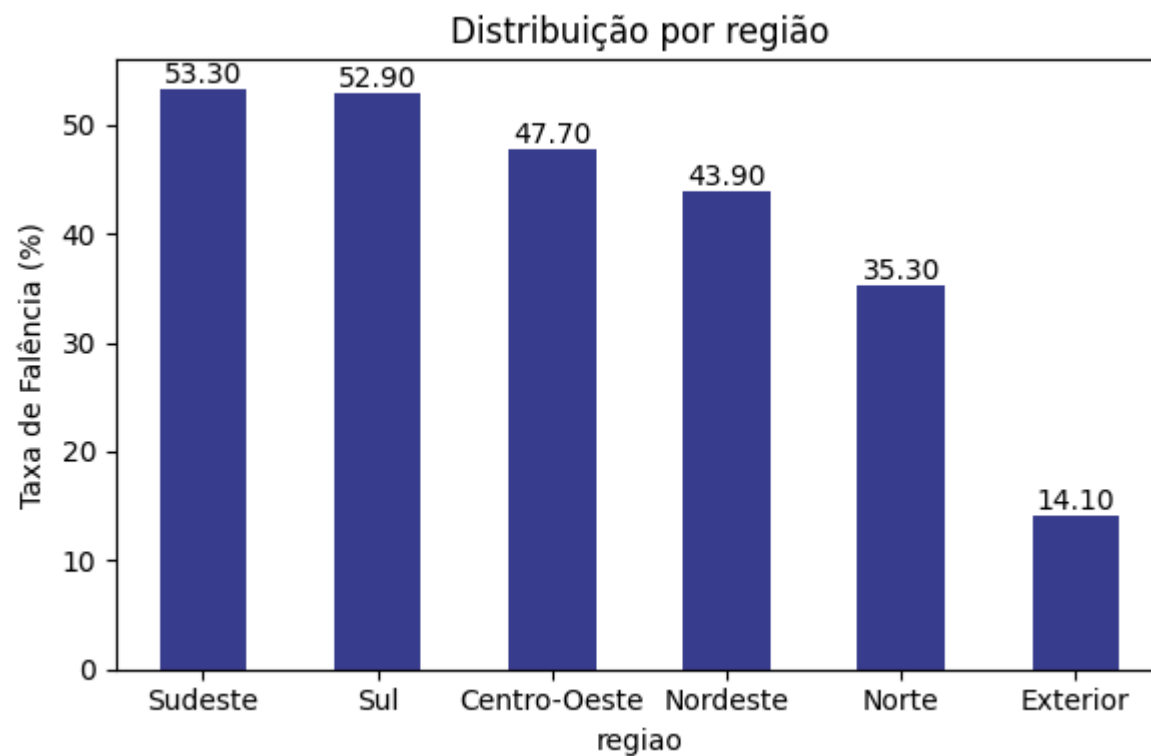
```
In [277... df_stack = (  
    df_analise  
    .groupby(["regiao", "status"])  
    .size()  
    .unstack(fill_value=0)  
)  
  
# Calcular taxa de falência  
df_stack["taxa_falencia"] = round((df_stack["FALIU"] / df_stack.sum(axis=1))*100, 1)
```

```
# Ordenar do maior para o menor
df_stack = df_stack.sort_values(by="taxa_falencia", ascending=False)

df_stack = df_stack.reset_index()

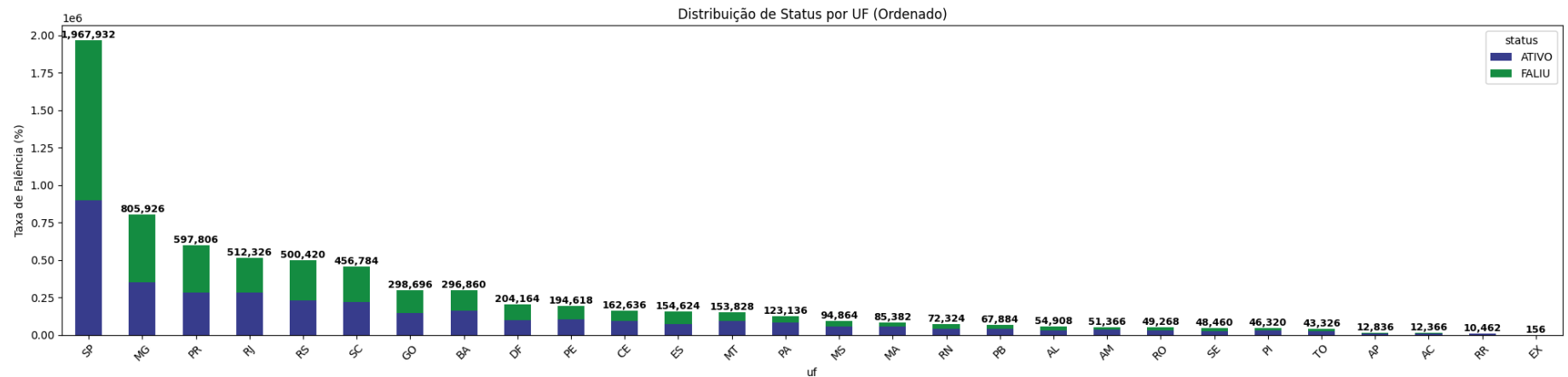
df_stack = df_stack.drop(columns=["ATIVO", "FALIU"])

plot_barra(
    df=df_stack,
    eixo_x="regiao",
    coluna_valor="taxa_falencia",
    aggfunc="mean",
    ylabel="Taxa de Falência (%)",
    titulo="Distribuição por região",
    paleta=PALETA
)
```

Verificando a taxa de falência para cada estado UF.

```
In [278... plot_barra_empilhada(  
    df=df_analise,  
    eixo_x="uf",  
    coluna_stack="status",  
    figsize=(20, 5),  
    ylabel="Taxa de Falência (%)",  
    titulo="Distribuição de Status por UF (Ordenado)",  
    paleta=PALETA  
)
```



In [279...

```
# Agrupamento
df_stack = (
    df_analise
    .groupby(["uf", "status"])
    .size()
    .unstack(fill_value=0)
)

# Calcular taxa de falência
df_stack["taxa_falencia"] = round((df_stack["FALIU"] / df_stack.sum(axis=1))*100, 2)

# Ordenar do maior para o menor
df_stack = df_stack.sort_values(by="taxa_falencia", ascending=False)

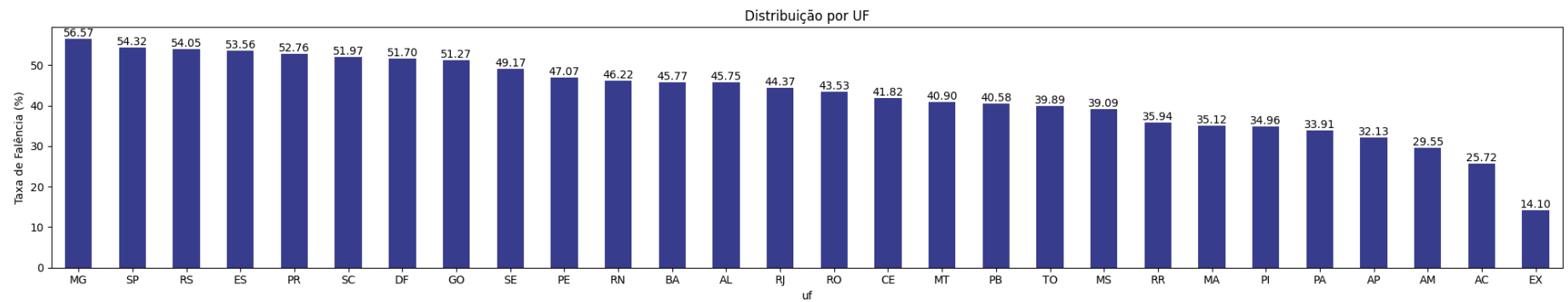
df_stack = df_stack.reset_index()

df_stack = df_stack.drop(columns=["ATIVO", "FALIU"])

plot_barra(
    df=df_stack,
    eixo_x="uf",
    coluna_valor="taxa_falencia",
    aggfunc="mean",
    figsize=(20, 4),
    ylabel="Taxa de Falência (%)",
    titulo="Distribuição por UF",
)
```

paleta=PALETA

)



In [280...

```
# Agrupamento
df_stack = (
    df_analise
    .groupby(["municipio_ident", "status"])
    .size()
    .unstack(fill_value=0)
)

# criar total (soma de todos os status)
df_stack["total"] = df_stack.sum(axis=1)

# ordenar corretamente
df_stack = df_stack.sort_values(by="total", ascending=False).head(20)

# Calcular taxa de falência
df_stack["taxa_falencia"] = round((df_stack["FALIU"] / df_stack.sum(axis=1))*100, 2)

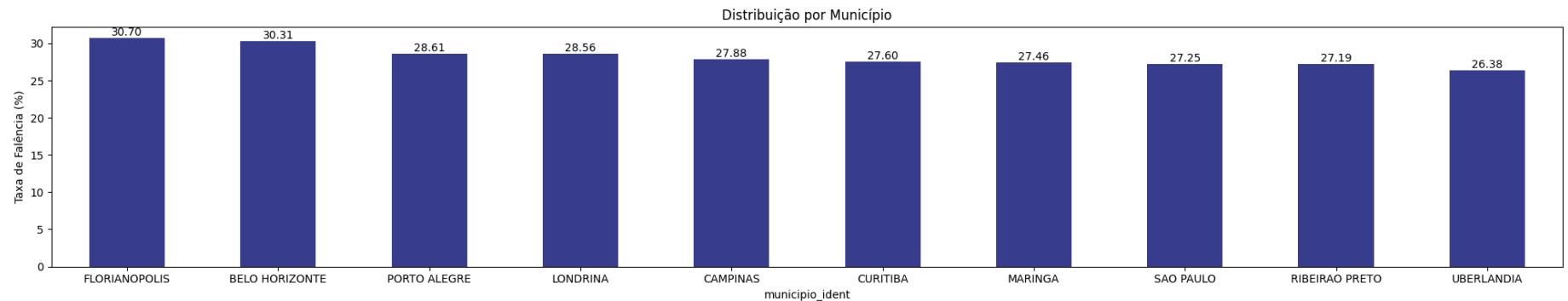
# Ordenar do maior para o menor
df_stack = df_stack.sort_values(by="taxa_falencia", ascending=False)

df_stack = df_stack.reset_index()

df_stack = df_stack.drop(columns=["ATIVO", "FALIU"])

plot_barra(
    df=df_stack,
```

```
eixo_x="municipio_ident",
coluna_valor="taxa_falencia",
aggfunc="mean",
top_n=10,
ylabel="Taxa de Falência (%)",
figsize=(20, 4),
titulo="Distribuição por Município",
paleta=PALETA
)
```



Empresas mais jovens falem mais?

Sim, empresas mais jovens tendem a ter a maior possibilidade de falência, tendo em média 54% de chance para empresas de 3 a 10 de funcionamento.

In [281...

```
# Criando uma faixa de idade para as empresas
bins = [0, 1, 3, 5, 10, 20, 50, 100]
labels = ["0-1", "1-3", "3-5", "5-10", "10-20", "20-50", "50+"]

df_analise["faixa_idade"] = pd.cut(df_analise["tempo_funcionamento"], bins=bins, labels=labels)

df_idade = (
    df_analise
    .groupby(["faixa_idade", "status"])
    .size()
    .unstack(fill_value=0)
)
```

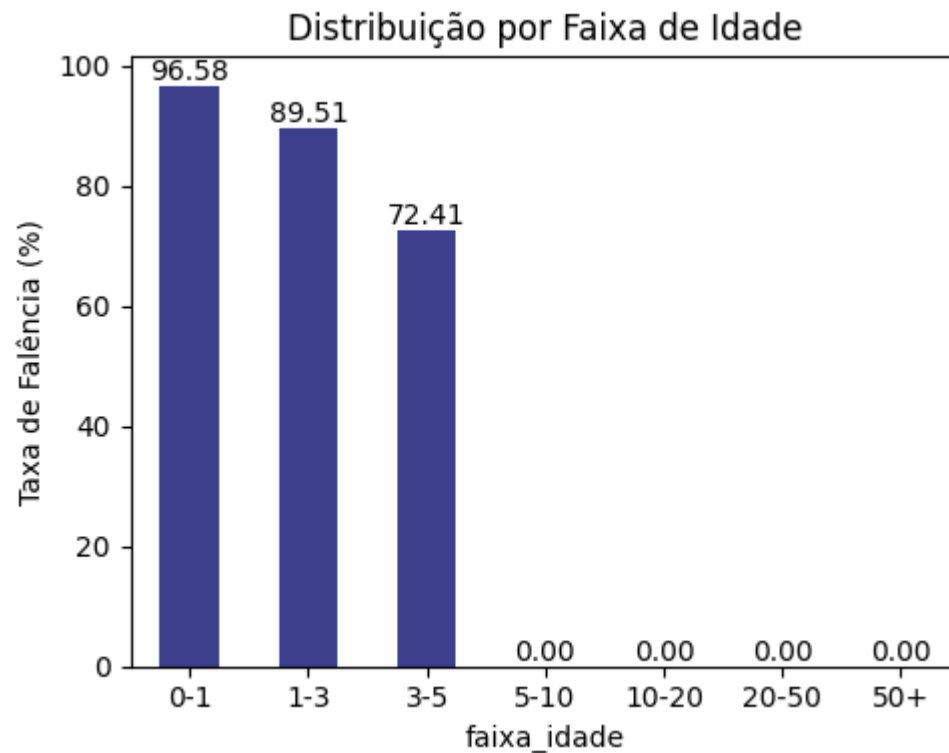
```
df_idade["taxa_falencia"] = (  
    round(df_idade["FALIU"] / (df_idade["FALIU"] + df_idade["ATIVO"]) * 100, 2)  
)  
  
df_plot = df_idade.sort_values(by="taxa_falencia", ascending=False)  
  
df_plot = df_plot.reset_index()  
  
df_plot = df_plot.drop(columns=["ATIVO", "FALIU"])  
  
plot_barra(  
    df=df_plot,  
    eixo_x="faixa_idade",  
    coluna_valor="taxa_falencia",  
    aggfunc="mean",  
    top_n=10,  
    ylabel="Taxa de Falência (%)",  
    figsize=(5, 4),  
    titulo="Distribuição por Faixa de Idade",  
    paleta=PALETA  
)
```

C:\Users\leona\AppData\Local\Temp\ipykernel_16796\129602199.py:9: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

```
.groupby(["faixa_idade", "status"])
```

C:\Users\leona\AppData\Local\Temp\ipykernel_16796\829801321.py:171: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

```
.groupby(eixo_x)[coluna_valor]
```



Capital social influencia a sobrevivência?

In [282...

```
df_capital = (  
    df_analise  
    .groupby(["faixa_capital", "status"])  
    .size()  
    .unstack(fill_value=0)  
)  
  
df_capital = df_capital.reindex(columns=["ATIVO", "FALIUI"], fill_value=0)  
  
df_capital["taxa_falencia"] = round((df_capital["FALIUI"] / (df_capital["FALIUI"] + df_capital["ATIVO"]))*100, 2)  
  
df_plot = df_capital.sort_values(by="taxa_falencia", ascending=False)  
  
df_plot = df_plot.reset_index()
```

```
df_plot = df_plot.drop(columns=["ATIVO", "FALIU"])

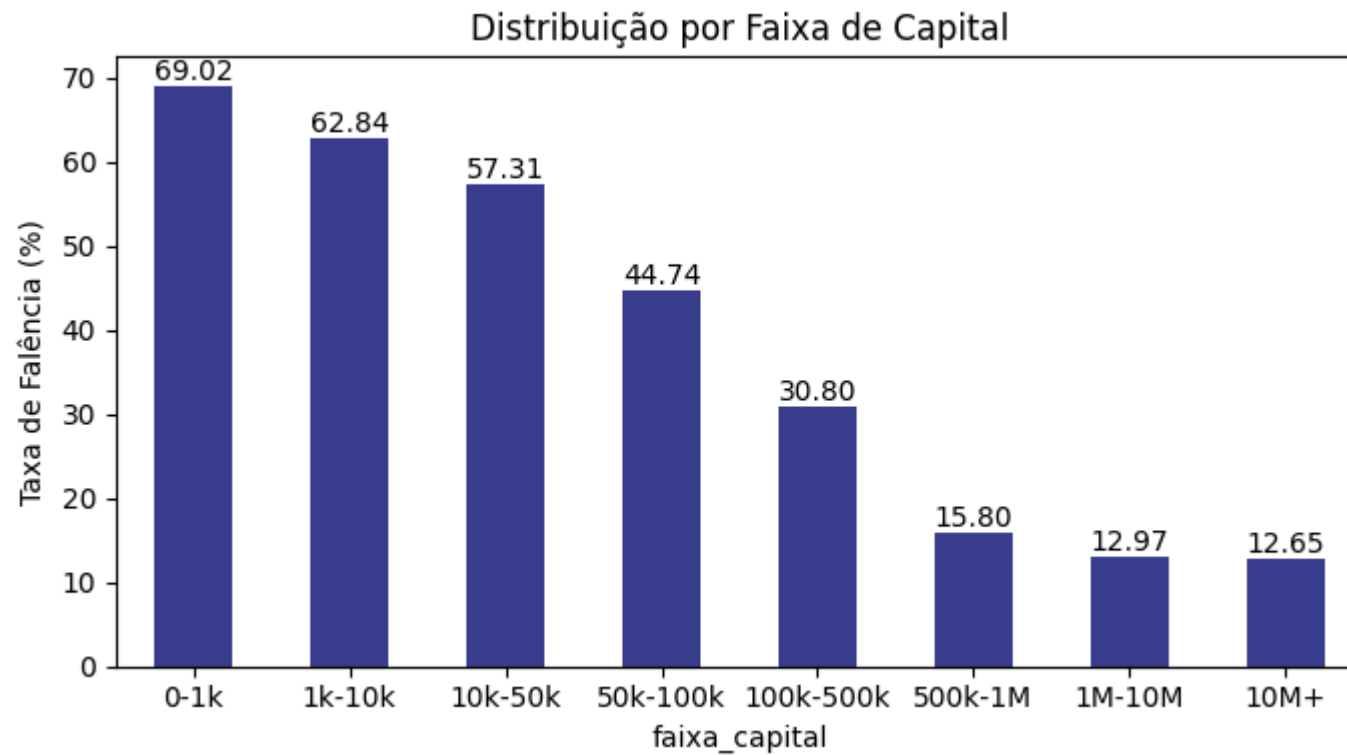
plot_barra(
    df=df_plot,
    eixo_x="faixa_capital",
    coluna_valor="taxa_falencia",
    aggfunc="mean",
    top_n=10,
    ylabel="Taxa de Falência (%)",
    figsize=(7, 4),
    titulo="Distribuição por Faixa de Capital",
    paleta=PALETA
)
```

C:\Users\leona\AppData\Local\Temp\ipykernel_16796\3231710533.py:3: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

```
.groupby(["faixa_capital", "status"])
```

C:\Users\leona\AppData\Local\Temp\ipykernel_16796\829801321.py:171: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

```
.groupby(eixo_x)[coluna_valor]
```



Quanto menor o capital social, maior a quantidade de empresas que faliram.

```
In [283... df_cross = (  
    df_analise  
    .groupby(["faixa_idade", "faixa_capital", "status"])  
    .size()  
    .unstack(fill_value=0)  
)  
  
df_cross["taxa_falencia"] = (  
    round((df_cross["FALIU"] / (df_cross["FALIU"] + df_cross["ATIVO"])) * 100, 2)  
)  
  
df_cross = df_cross.reset_index()
```



```
# selecionar colunas
df_cross = df_cross[["faixa_idade", "faixa_capital", "taxa_falencia"]]
```

C:\Users\leona\AppData\Local\Temp\ipykernel_16796\187238797.py:3: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

```
.groupby(["faixa_idade", "faixa_capital", "status"])
```

Vamos analisar o mapa de calor entre faixa de idade e faixa de capital.

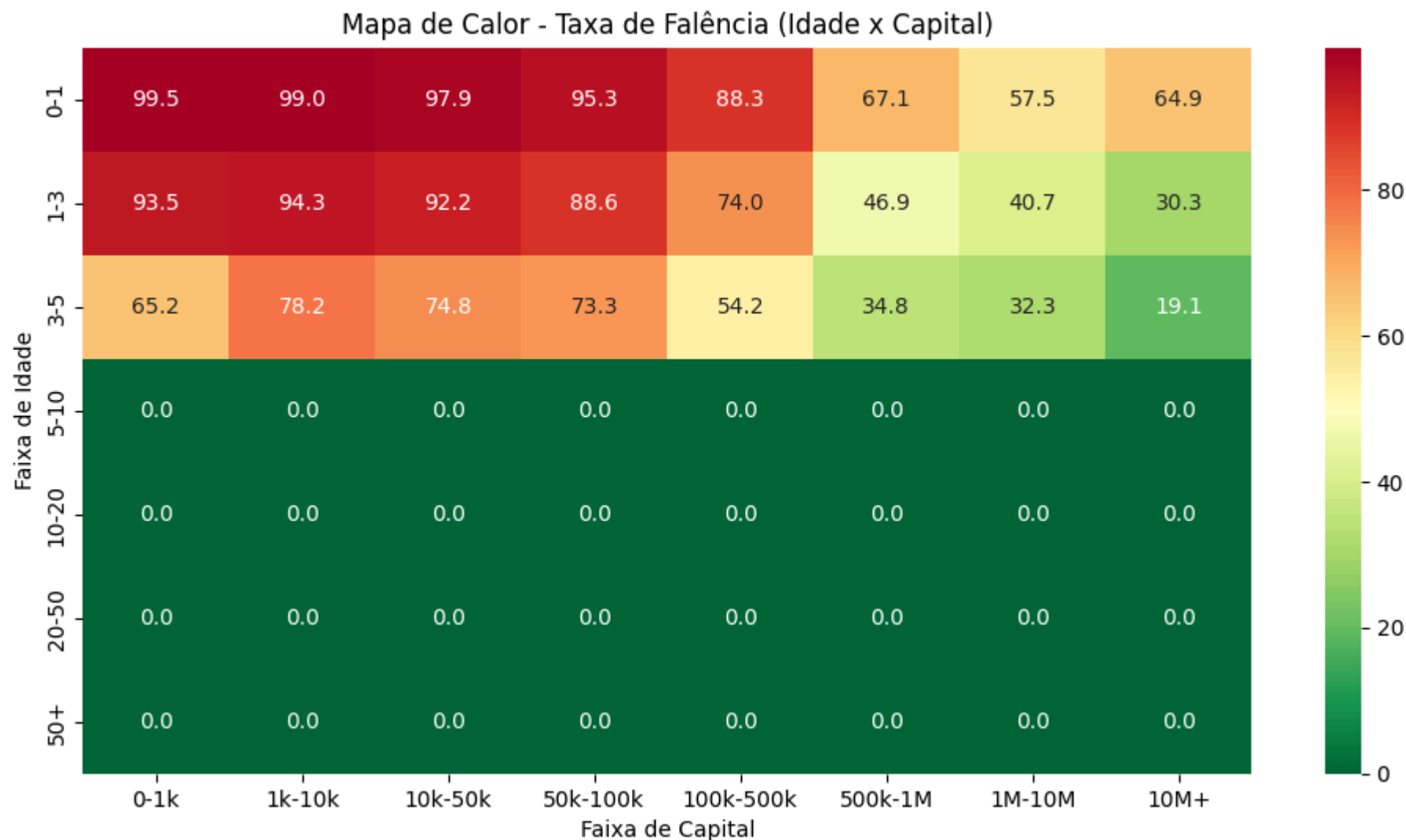
```
In [284... df_heatmap = df_cross.pivot(
    index="faixa_idade",
    columns="faixa_capital",
    values="taxa_falencia"
)

plt.figure(figsize=(12, 6))

sns.heatmap(
    df_heatmap,
    annot=True,          # mostra valores
    fmt=".1f",           # formato (percentual sem %)
    cmap="RdYlGn_r",     # vermelho = ruim, verde = bom
)

plt.title("Mapa de Calor - Taxa de Falência (Idade x Capital)")
plt.xlabel("Faixa de Capital")
plt.ylabel("Faixa de Idade")

plt.show()
```



Para empresas recém criadas até 20 anos de existência, a marca de 100k de capital determina uma diminuição significativa na probabilidade de falência. Quanto mais jovem e menor o capital social da empresa maior a probabilidade dela falir, sendo o primeiro ano primordial para a continuidade da empresa.

2.3 Analisando influência da faixa etária e de capital a região específica

Para essa análise veremos apenas as regiões Sudeste, Nordeste e Norte, para verificar as influências para a região com a maior taxa e para as regiões com menor taxa.

2.3.1 Sudeste

```
In [285... df_analise2 = df_analise[df_analise['regiao'] == 'Sudeste'].copy()

df_cross = (
    df_analise2
    .groupby(["faixa_idade", "faixa_capital", "status"])
    .size()
    .unstack(fill_value=0)
)

df_cross["taxa_falencia"] = (
    round((df_cross["FALIU"] / (df_cross["FALIU"] + df_cross["ATIVO"])) * 100, 2)
)

df_cross = df_cross.reset_index()

# selecionar colunas
df_cross = df_cross[["faixa_idade", "faixa_capital", "taxa_falencia"]]

df_heatmap = df_cross.pivot(
    index="faixa_idade",
    columns="faixa_capital",
    values="taxa_falencia"
)

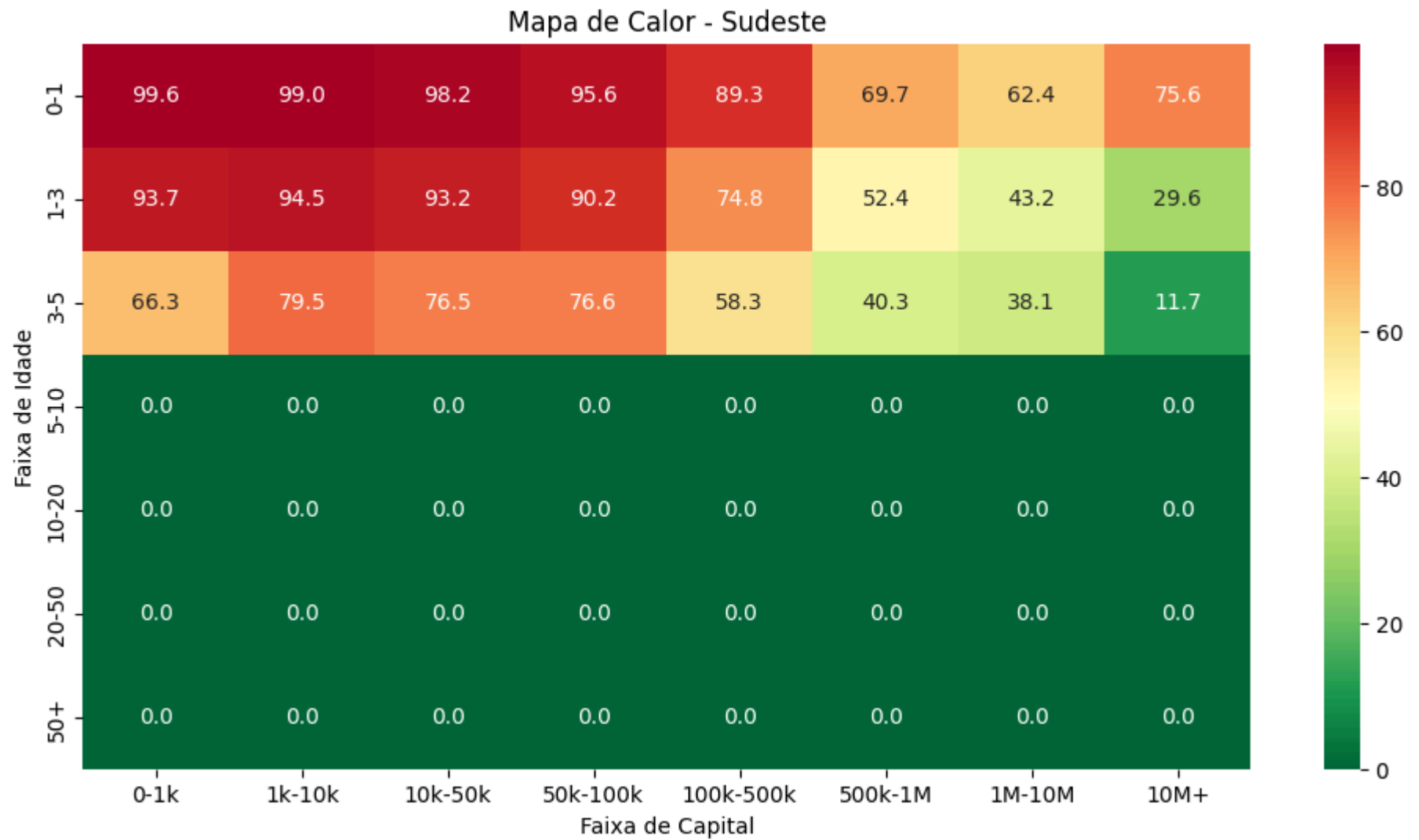
plt.figure(figsize=(12, 6))

sns.heatmap(
    df_heatmap,
    annot=True,          # mostra valores
    fmt=".1f",           # formato (percentual sem %)
    cmap="RdYlGn_r",     # vermelho = ruim, verde = bom
)

plt.title("Mapa de Calor - Sudeste")
```

```
plt.xlabel("Faixa de Capital")  
plt.ylabel("Faixa de Idade")  
  
plt.show()
```

C:\Users\leona\AppData\Local\Temp\ipykernel_16796\4243553966.py:5: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.
.groupby(["faixa_idade", "faixa_capital", "status"])



2.3.2 Nordeste

```
In [286... df_analise2 = df_analise[df_analise['regiao'] == 'Nordeste'].copy()

df_cross = (
    df_analise2
    .groupby(["faixa_idade", "faixa_capital", "status"])
```

```
.size()
.unstack(fill_value=0)
)

df_cross["taxa_falencia"] = (
    round((df_cross["FALIU"] / (df_cross["FALIU"] + df_cross["ATIVO"]))* 100, 2)
)

df_cross = df_cross.reset_index()

# selecionar colunas
df_cross = df_cross[["faixa_idade", "faixa_capital", "taxa_falencia"]]

df_heatmap = df_cross.pivot(
    index="faixa_idade",
    columns="faixa_capital",
    values="taxa_falencia"
)

plt.figure(figsize=(12, 6))

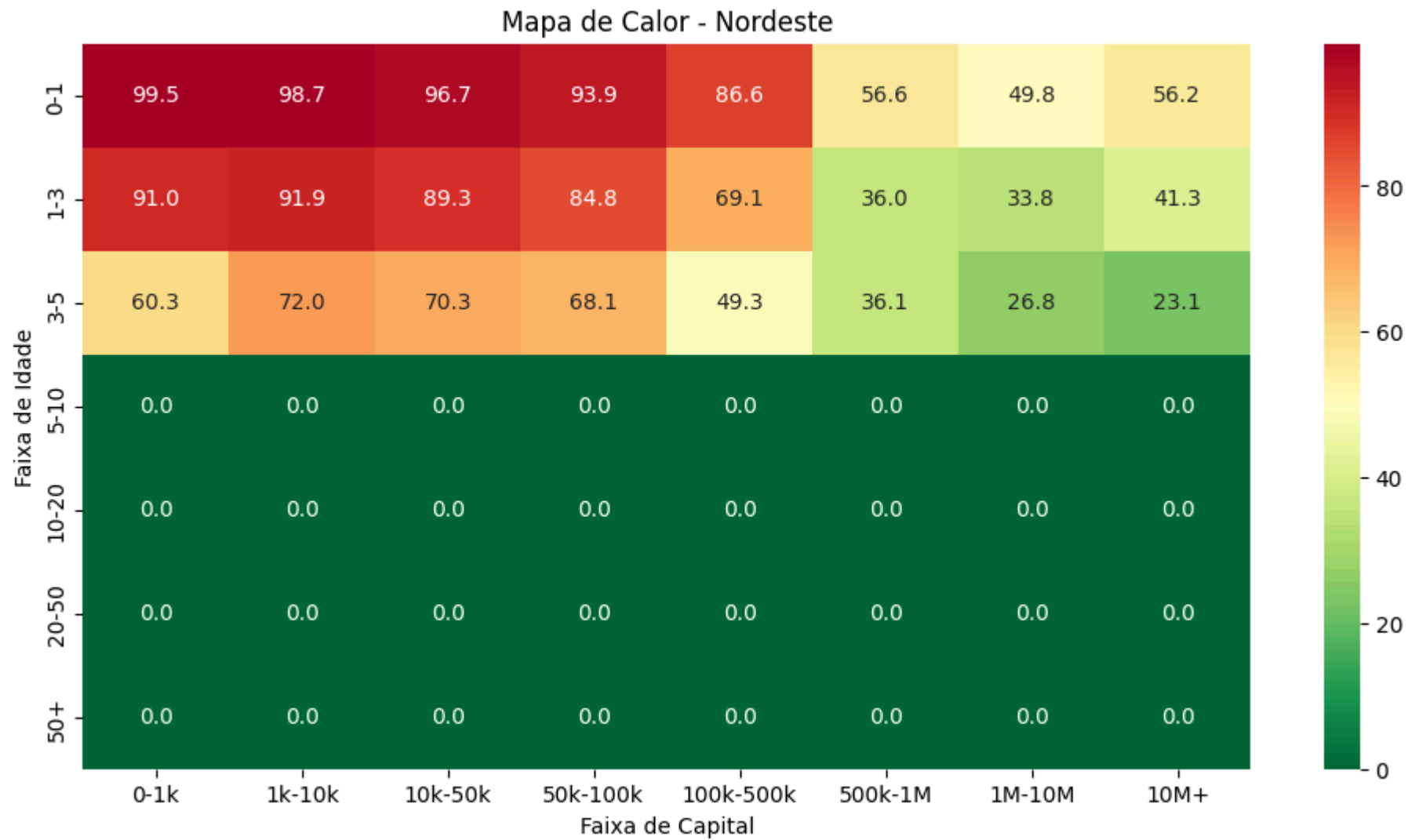
sns.heatmap(
    df_heatmap,
    annot=True,          # mostra valores
    fmt=".1f",           # formato (percentual sem %)
    cmap="RdYlGn_r",     # vermelho = ruim, verde = bom
)

plt.title("Mapa de Calor - Nordeste")
plt.xlabel("Faixa de Capital")
plt.ylabel("Faixa de Idade")

plt.show()
```

C:\Users\leona\AppData\Local\Temp\ipykernel_16796\3670770029.py:5: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

```
.groupby(["faixa_idade", "faixa_capital", "status"])
```



2.3.3 Norte

```
In [287... df_analise2 = df_analise[df_analise['regiao'] == 'Norte'].copy()

df_cross = (
    df_analise2
    .groupby(["faixa_idade", "faixa_capital", "status"])
```

```
.size()
.unstack(fill_value=0)
)

df_cross["taxa_falencia"] = (
    round((df_cross["FALIU"] / (df_cross["FALIU"] + df_cross["ATIVO"]))* 100, 2)
)

df_cross = df_cross.reset_index()

# selecionar colunas
df_cross = df_cross[["faixa_idade", "faixa_capital", "taxa_falencia"]]

df_heatmap = df_cross.pivot(
    index="faixa_idade",
    columns="faixa_capital",
    values="taxa_falencia"
)

plt.figure(figsize=(12, 6))

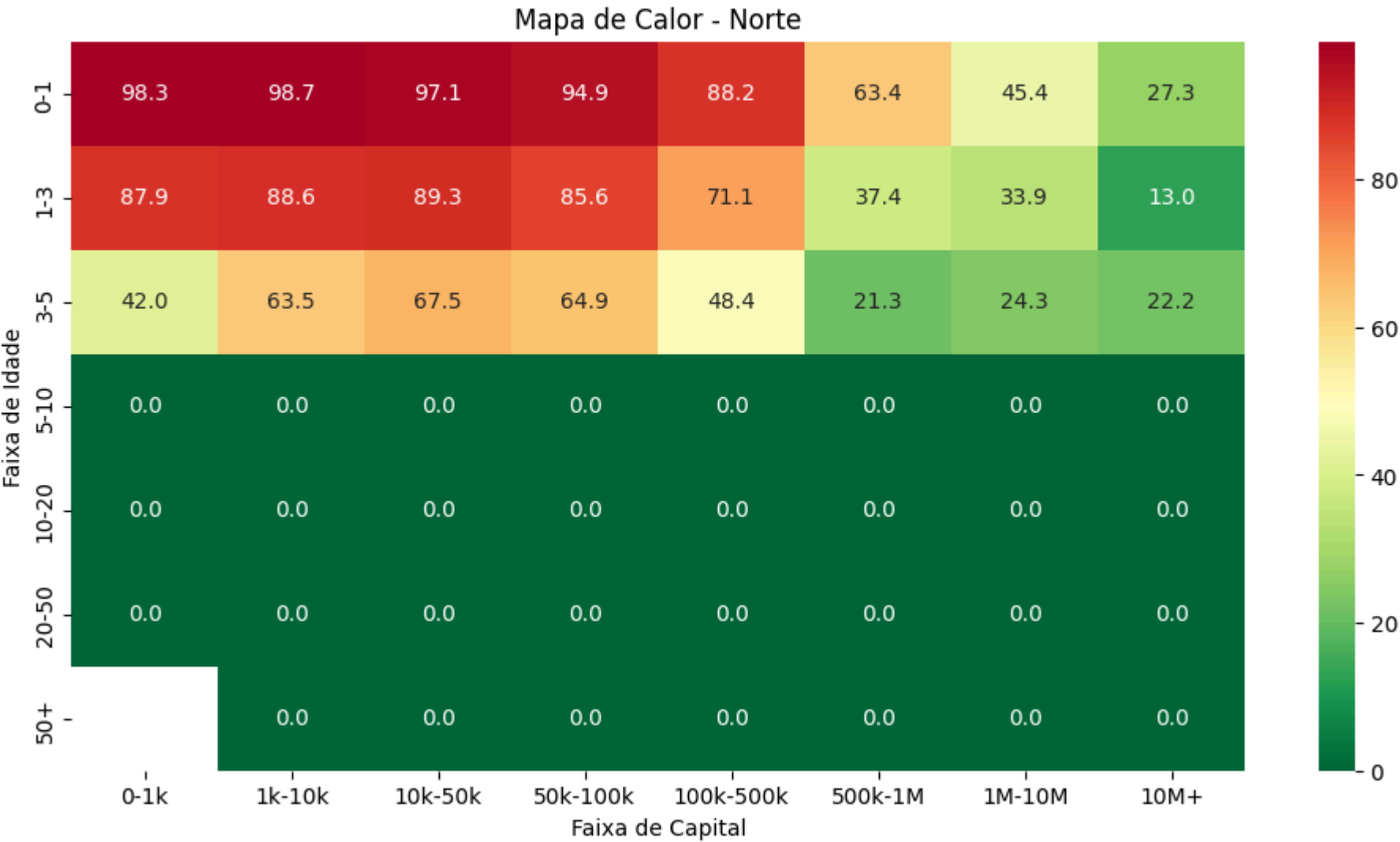
sns.heatmap(
    df_heatmap,
    annot=True,          # mostra valores
    fmt=".1f",           # formato (percentual sem %)
    cmap="RdYlGn_r",     # vermelho = ruim, verde = bom
)

plt.title("Mapa de Calor - Norte")
plt.xlabel("Faixa de Capital")
plt.ylabel("Faixa de Idade")

plt.show()
```

C:\Users\leona\AppData\Local\Temp\ipykernel_16796\901422296.py:5: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

```
.groupby(["faixa_idade", "faixa_capital", "status"])
```

Para a análise por regiões obtemos os seguintes insights:

- Vemos aqui a manutenção da regra geral descoberta anteriormente, o primeiro ano é um marco essencial para a continuidade da empresa, tal perspectiva só é quebrada de acordo com o capital;
- Quanto mais antiga a empresa menores os riscos de falência;

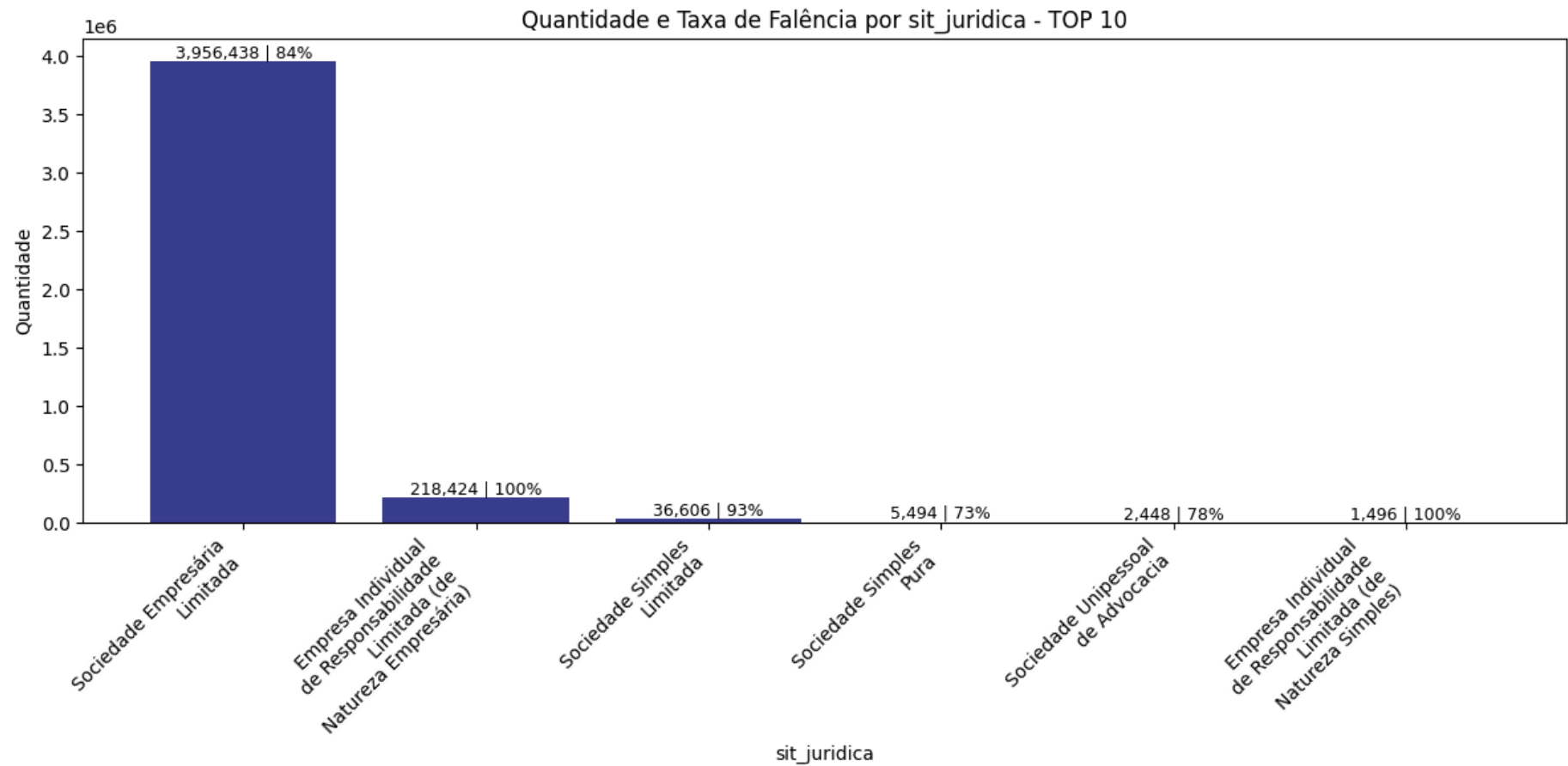
- Para a região sudeste temos uma maior estabilidade após as marcas de 10 anos e capital superior a 100k em comparação a outras regiões, no entanto o risco ao iniciar o negócio é muito maior, tal comportamento nos revela um ambiente mais propício para empresas de tradição e maior capital;
- Para a região nordeste vemos muita variabilidade demonstrando que o capital auxilia mas não é um definidor, para empresas de 3-5 anos temos as taxas de falência mais perigosas denotando um momento perigoso no crescimento da empresa;
- Para a região norte vemos um comportamento parecido a região nordeste, no entanto, a faixa etária das empresas diminuiu determinando um novo marco para melhora de possibilidade de continuidade da empresa, ou seja, menor tempo para aumentar as chances de continuidade da empresa.

2.4 Analisando influência da situação jurídica

Neste marco analisaremos essa influência para as empresas de até 5 anos de existência por ser o alvo deste estudo.

```
In [288... df_ate_5 = df_analise[df_analise["tempo_funcionamento"] <= 5].copy()
```

```
In [289... plot_barra_falencia(  
    df_ate_5,  
    eixo_x="sit_juridica",  
    coluna_status="status",  
    status_ativo="ATIVO",  
    status_faliu="FALIU",  
    minimo_registros=1000,  
    top_n=10,  
    pareto=False,  
    largura_texto=20,  
    figsize=(12, 6),  
    titulo=None,  
    paleta=PALETA  
)
```



Out[289...

status	sit_juridica	ATIVO	FALIU	total	taxa_falencia
0	Sociedade Empresária\nLimitada	641610	3314828	3956438	0.837831
1	Empresa Individual\nde Responsabilidade\nLimit...	0	218424	218424	1.000000
2	Sociedade Simples\nLimitada	2404	34202	36606	0.934328
3	Sociedade Simples\nPura	1470	4024	5494	0.732435
4	Sociedade Unipessoal\nde Advocacia	532	1916	2448	0.782680
5	Empresa Individual\nde Responsabilidade\nLimit...	0	1496	1496	1.000000

Considerando o volume acima de 2.000 como significativo vemos uma das naturezas jurídicas dá certeza de falência que seria *Empresa Individual de Responsabilidade Limitada*.

Outras naturezas jurídicas são indicadores de aumento de probabilidade de falência. Seriam eles:

- Sociedade Unipessoal de Advocacia
- Sociedade Empresária Limitada

E existem algumas naturezas jurídicas que diminuem a probabilidade de falência em até 5 anos:

- Sociedade Simples Pura
- Sociedade Simples Limitada

2.5 Analisando influência da dos tempos tempo_op_simples, tempo_excl_op_simples e tempo_exclusao_mei

Vamos analisar este tópico na perspectiva das empresas que não faliram dentro de 5 anos para verificar algum comportamento positivo.

```
In [290...] df_ate_5_status = df_analise[(df_analise["tempo_funcionamento"] <= 5) & (df_analise['status'] == 'ATIVO')].copy()
```

- Faixa de Capital

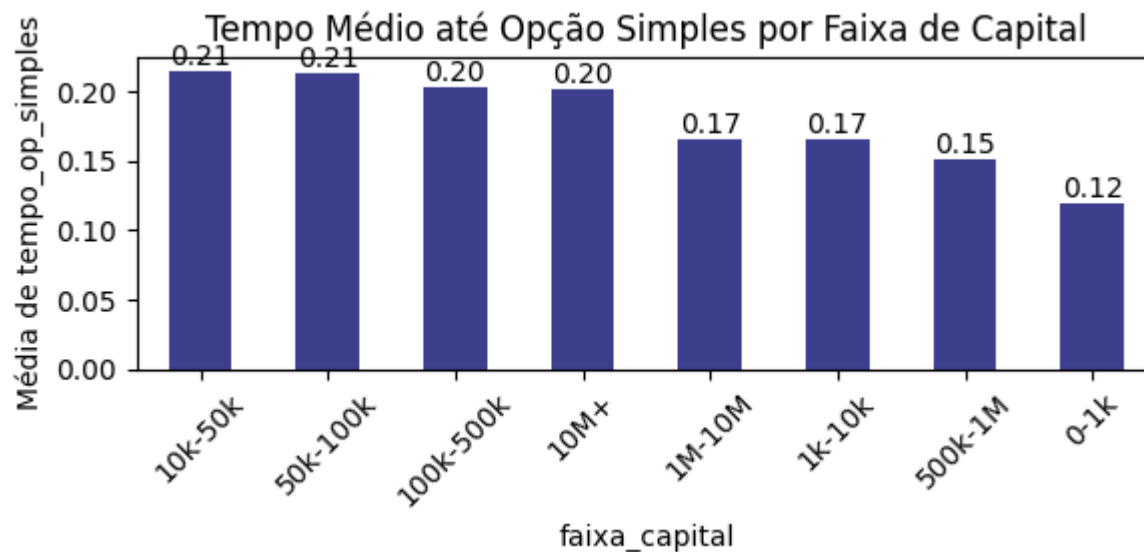
```
In [291...] plot_media(  
    df_ate_5_status,  
    "tempo_op_simples",  
    "faixa_capital",  
    "Tempo Médio até Opção Simples por Faixa de Capital",  
    paleta=PALETA  
)  
  
plot_media(  
    df_ate_5_status,  
    "tempo_excl_op_simples",  
    "faixa_capital",  
    "Tempo Médio até Exclusão do Simples por Faixa de Capital",
```

```
paleta=PALETA
)

plot_media(
    df_ate_5_status,
    "tempo_exclusao_mei",
    "faixa_capital",
    "Tempo Médio até Exclusão MEI por Faixa de Capital",
    paleta=PALETA
)
```

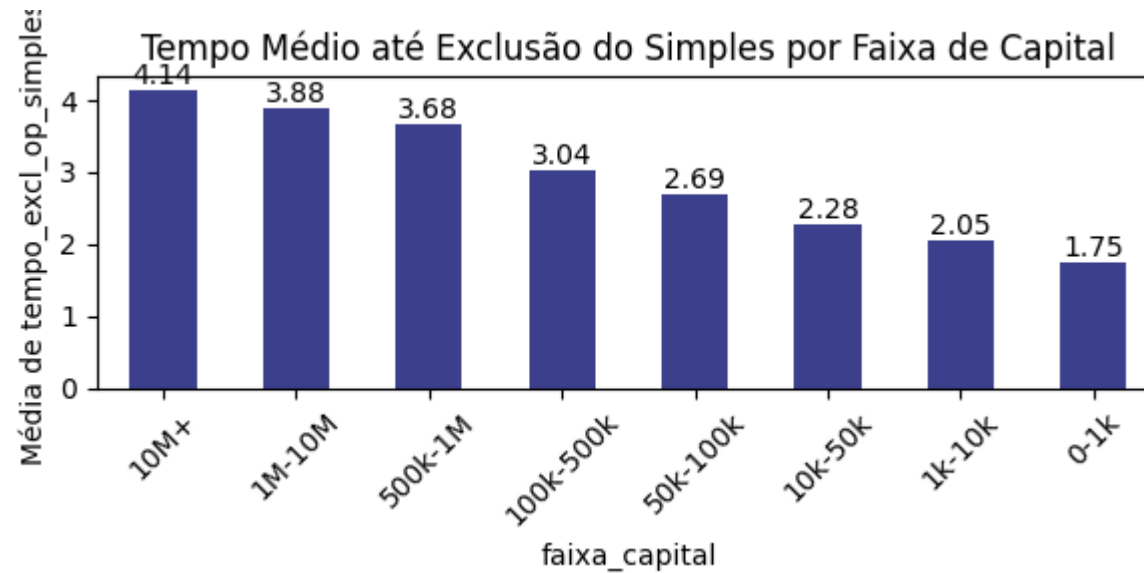
C:\Users\leona\AppData\Local\Temp\ipykernel_16796\829801321.py:26: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

```
.groupby(group_col)[coluna]
```



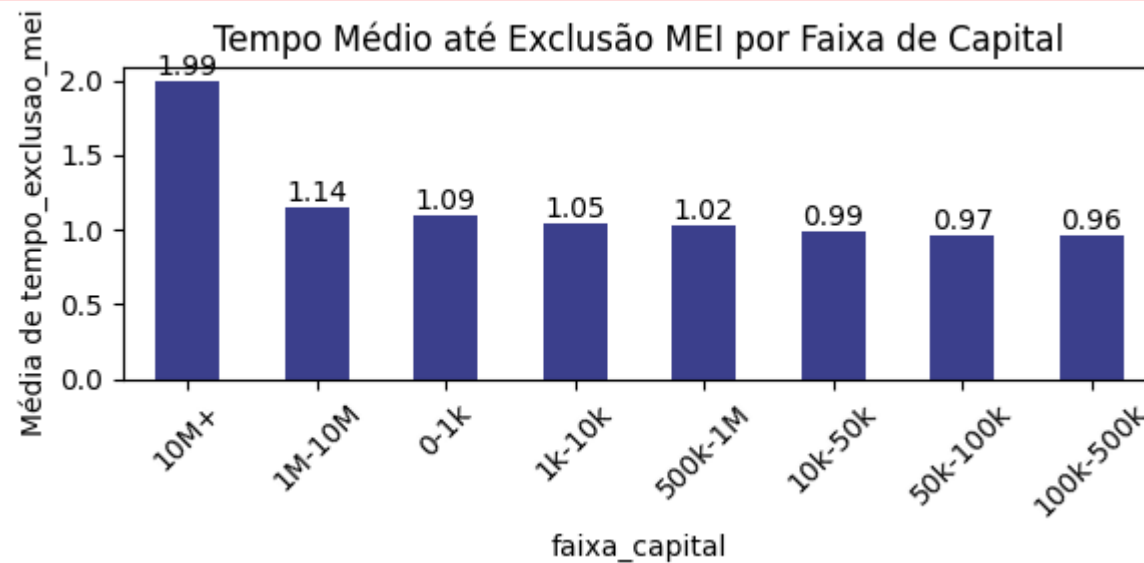
C:\Users\leona\AppData\Local\Temp\ipykernel_16796\829801321.py:26: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

```
.groupby(group_col)[coluna]
```



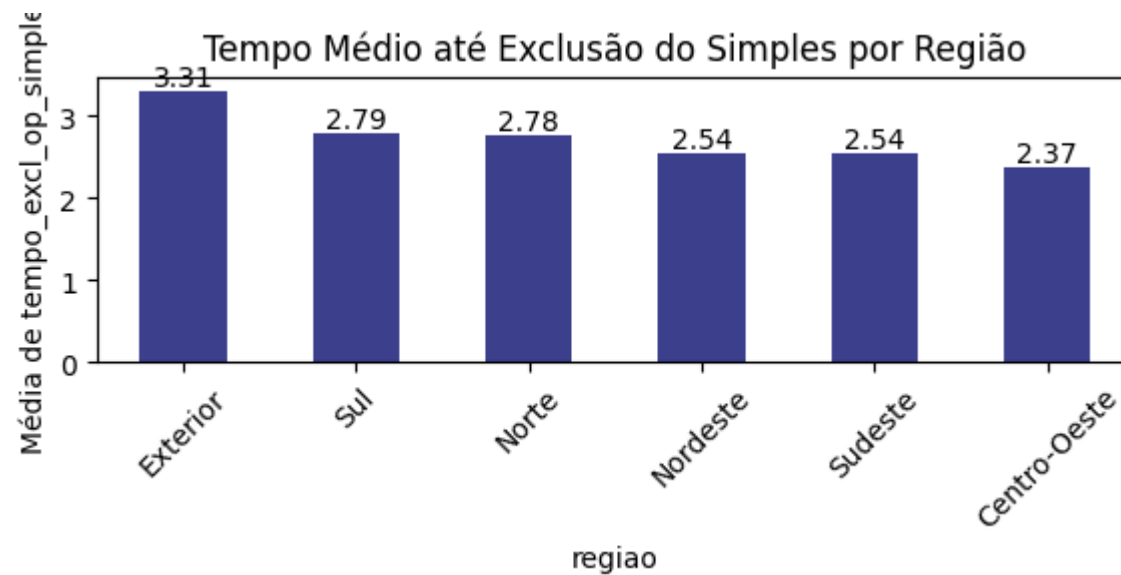
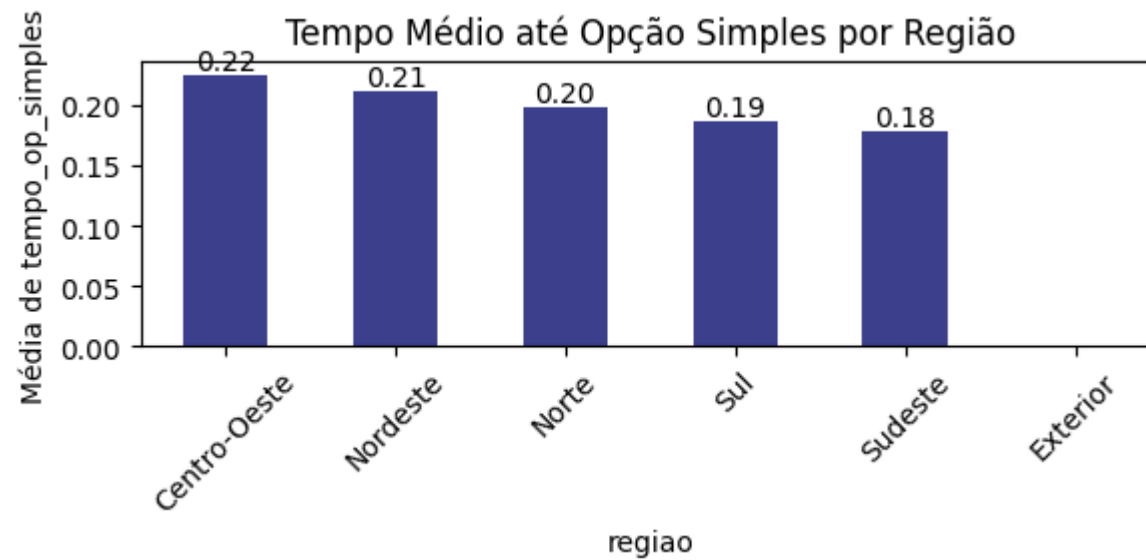
C:\Users\leona\AppData\Local\Temp\ipykernel_16796\829801321.py:26: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

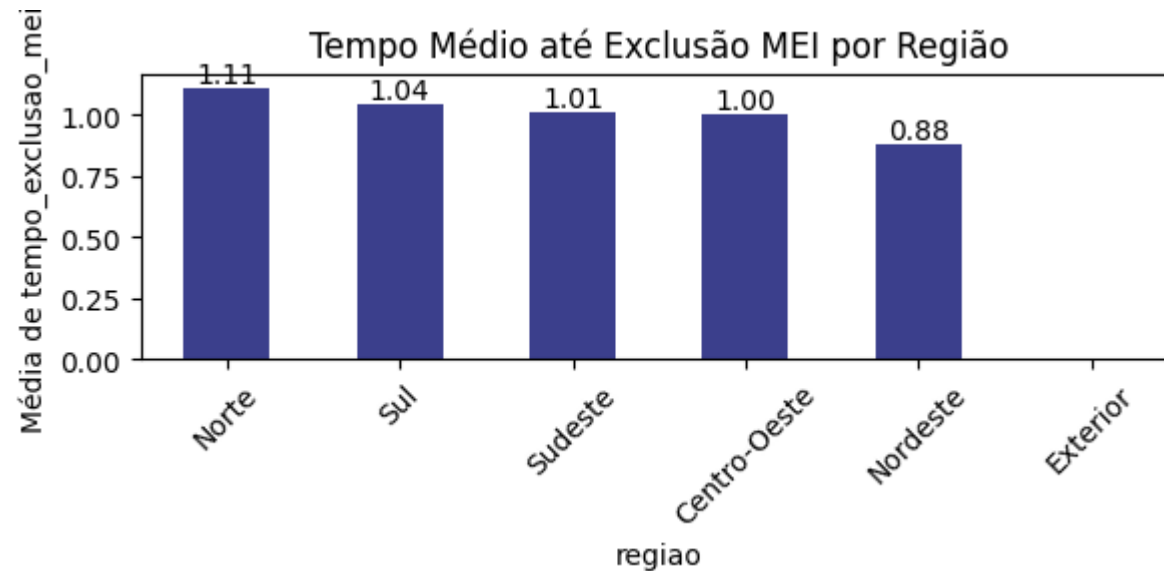
.groupby(group_col)[coluna]



- Região

```
In [292... plot_media(  
    df_ate_5_status,  
    "tempo_op_simples",  
    "regiao",  
    "Tempo Médio até Opção Simples por Região",  
    paleta=PALETA  
)  
  
plot_media(  
    df_ate_5_status,  
    "tempo_excl_op_simples",  
    "regiao",  
    "Tempo Médio até Exclusão do Simples por Região",  
    paleta=PALETA  
)  
  
plot_media(  
    df_ate_5_status,  
    "tempo_exclusao_mei",  
    "regiao",  
    "Tempo Médio até Exclusão MEI por Região",  
    paleta=PALETA  
)
```





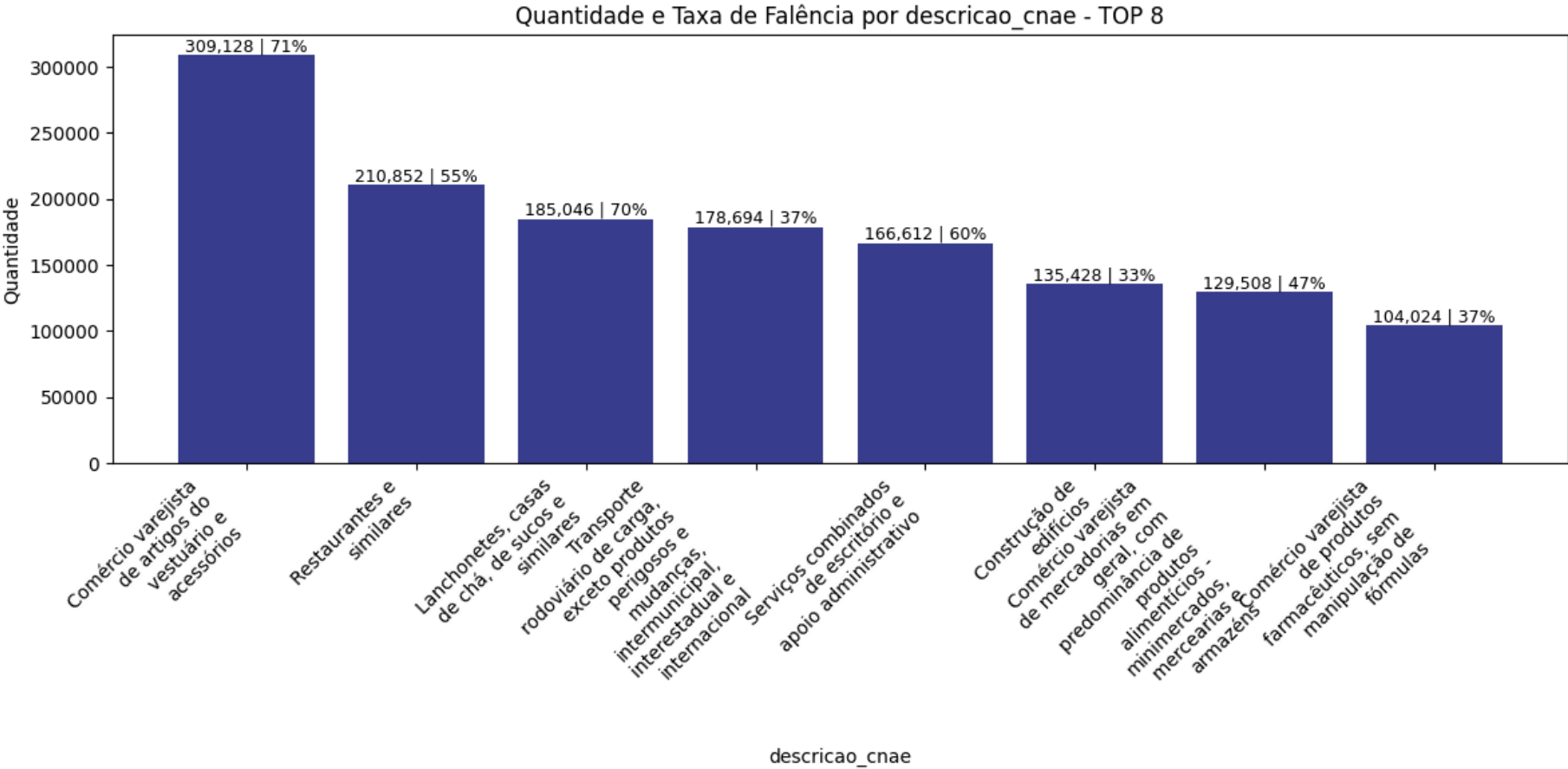
2.6 Analisando influência do CNAE (Código Nacional de Atividades Econômicas)

```
In [293...] df_ate_5 = df_analise[df_analise["tempo_funcionamento"] <= 5].copy()
```

- CNAE Principal

```
In [294...] returned_df = plot_barra_falencia(  
    df_analise,  
    eixo_x="descricao_cnae",  
    coluna_status="status",  
    status_ativo="ATIVO",  
    status_faliu="FALIU",  
    minimo_registros=10000,  
    top_n=8,  
    pareto=False,  
    largura_texto=20,  
    figsize=(12, 6),  
    titulo=None,  
    paleta=PALETA  
)
```

returned_df



Out [294...

	status	descricao_cnae	ATIVO	FALIU	total	taxa_falencia
0		Comércio varejista\nde artigos do\investuário e...	89512	219616	309128	0.710437
1		Restaurantes e\nsimilares	94744	116108	210852	0.550661
2		Lanchonetes, casas\nde chá, de sucos e\nsimilares	56228	128818	185046	0.696140
3		Transporte\nrodoviário de carga,\nexcto produ...	112660	66034	178694	0.369537
4		Serviços combinados\nde escritório e\apoio ad...	66136	100476	166612	0.603054
5		Construção de\nedifícios	90294	45134	135428	0.333269
6		Comércio varejista\nde mercadorias em\ngeral, ...	68174	61334	129508	0.473592
7		Comércio varejista\nde produtos\farmacêuticos...	65190	38834	104024	0.373318

In [295...

```

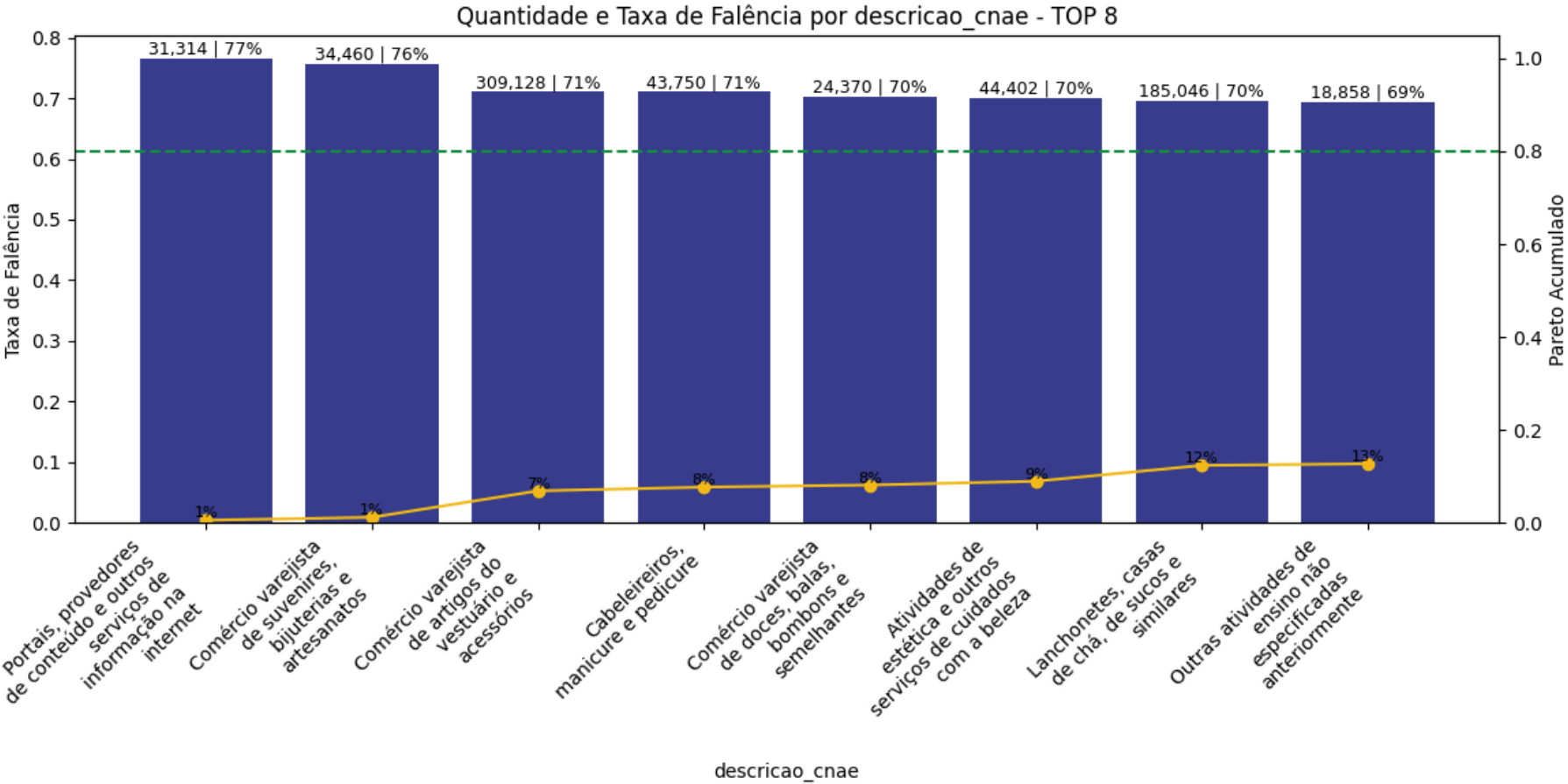
returned_df = plot_barra_falencia(
    df_analise,
    eixo_x="descricao_cnae",
    coluna_status="status",
    status_ativo="ATIVO",
    status_faliu="FALIU",
    minimo_registros=10000,
    top_n=8,
    pareto=True,
    largura_texto=20,
    figsize=(12, 6),
    titulo=None,
    paleta=PALETA
)

```

```

returned_df

```



Out [295...

	status	descricao_cnae	ATIVO	FALIU	total	taxa_falencia	pareto
0	Portais, provedores\nde conteúdo e outros\ner...		7338	23976	31314	0.765664	0.005765
1	Comércio varejista\nde suvenires,\nbijuterias ...		8366	26094	34460	0.757226	0.012109
2	Comércio varejista\nde artigos do\vestuário e...		89512	219616	309128	0.710437	0.069020
3	Cabeleireiros,\nmanicure e pedicure		12670	31080	43750	0.710400	0.077074
4	Comércio varejista\nde doces, balas,\nbombons ...		7230	17140	24370	0.703324	0.081561
5	Atividades de\nestética e outros\nserviços de ...		13280	31122	44402	0.700914	0.089735
6	Lanchonetes, casas\nde chá, de sucos e\nsimilares		56228	128818	185046	0.696140	0.123802
7	Outras atividades de\ne ensino não\nespecificada...		5778	13080	18858	0.693605	0.127274

In [296...

```

descricoes = returned_df['descricao_cnae'].tolist()

desc = [descricao.replace("\n", " ") for descricao in descricoes]

text_desc = "\n".join(desc)

print(f"As descrições do CNAE mais falidas representando 80% das incidencias são:\n\n{text_desc}")

```

As descrições do CNAE mais falidas representando 80% das incidencias são:

Portais, provedores de conteúdo e outros serviços de informação na internet
 Comércio varejista de suvenires, bijuterias e artesanatos
 Comércio varejista de artigos do vestuário e acessórios
 Cabeleireiros, manicure e pedicure
 Comércio varejista de doces, balas, bombons e semelhantes
 Atividades de estética e outros serviços de cuidados com a beleza
 Lanchonetes, casas de chá, de sucos e similares
 Outras atividades de ensino não especificadas anteriormente

In [299...

```

regiao = {"Norte":1, "Nordeste":2, "Centro-Oeste":3, "Sudeste":4, "Sul":5, "Exterior":6}

df_analise["regiao_num"] = df_analise["regiao"].map(regiao)

```

```
df_ate_5["regiao_num"] = df_ate_5["regiao"].map(regiao)

df_corr = df_ate_5.copy()

df_corr["target_faliu"] = (
    df_corr["situacao_cadastral"].astype(str) == "8"
).astype(int)

# COLUNAS DE INTERESSE
selected_cols = [
    "natureza_juridica",
    "regiao_num",
    "capital_social",
    "tempo_exclusao_mei",
    "tempo_op_simples",
    "tempo_excl_op_simples",
    "cnae_fiscal_principal",
    "municipio"
]

# manter apenas existentes
selected_cols = [col for col in selected_cols if col in df_corr.columns]

# garantir numérico
df_corr[selected_cols] = df_corr[selected_cols].apply(
    pd.to_numeric, errors="coerce"
)

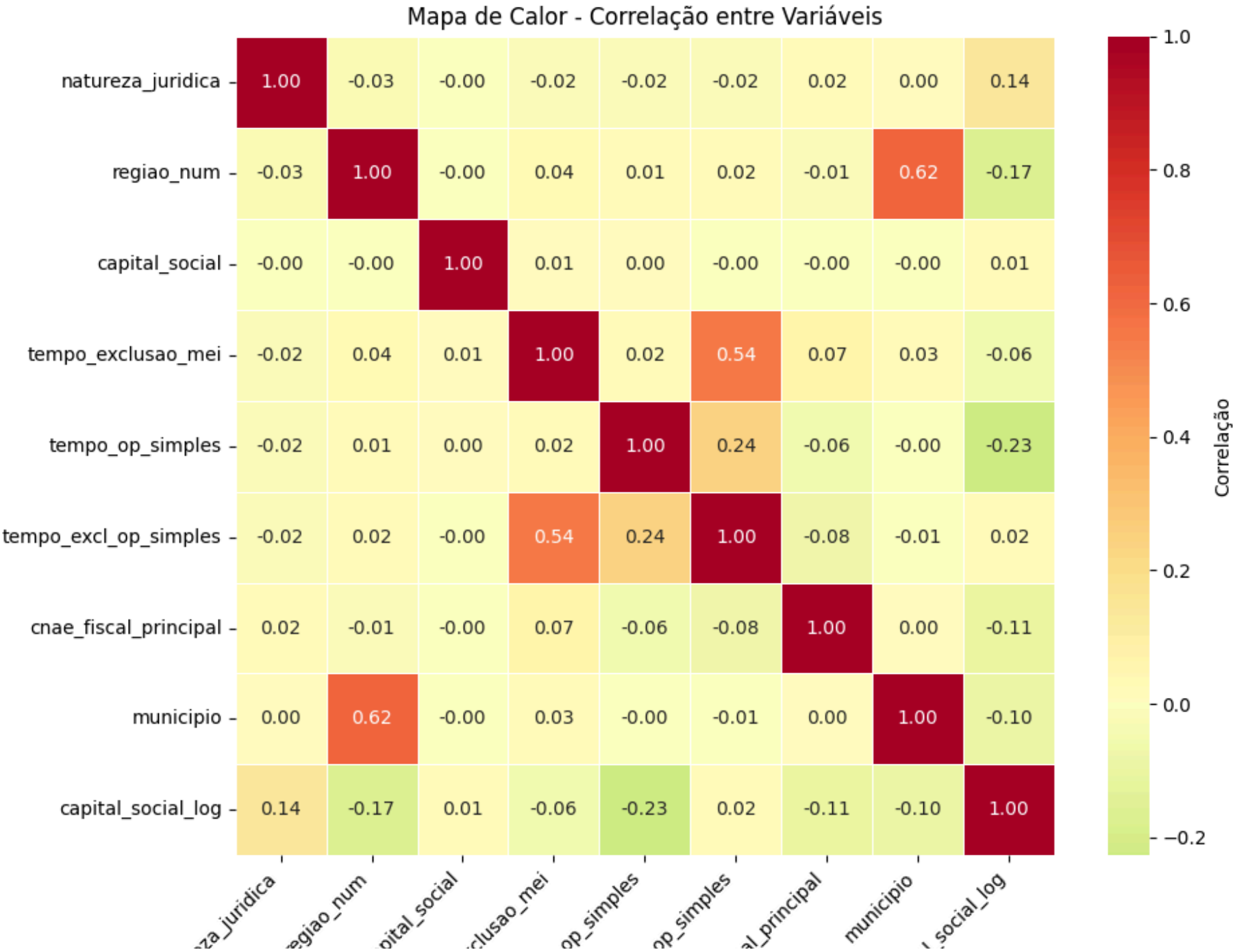
# (opcional) transformação log
if "capital_social" in df_corr.columns:
    df_corr["capital_social_log"] = np.log1p(df_corr["capital_social"])
    selected_cols.append("capital_social_log")

# MATRIZ DE CORRELAÇÃO
corr_matrix = df_corr[selected_cols].corr(numeric_only=True)

# PLOT
plt.figure(figsize=(10, 8))

sns.heatmap(
    corr_matrix,
```

```
annot=True,  
fmt=".2f",  
cmap="RdYlGn_r",  
center=0,  
square=True,  
linewidths=0.5,  
cbar_kws={"label": "Correlação"}  
)  
  
plt.title("Mapa de Calor - Correlação entre Variáveis")  
plt.xticks(rotation=45, ha="right")  
plt.yticks(rotation=0)  
  
plt.tight_layout()  
plt.show()
```





De acordo com esse gráfico por conta da correlação de 0.6, podemos remover o município do pré-processamento.

3. Conclusões

Durante a exploração de dados vimos que a taxa de falência foi 51% nos indicando que mais da metade das empresas registradas faliram no mês analisado, sendo que 54% das empresas que faliram eram micro-empresas, ou seja, menor capital e provavelmente empresas em início de trajetória.

Ao analisarmos a distribuição de empresas que faliram por estado, vimos uma concentração de resultados negativos em determiandas regiões indicando influência da localidade nos resultados.

Dessa forma foi traçado as seguintes premissas:

Premissa 1. O Ano 1 é o marco mais importante para a continuidade do negócio

A taxa de falência para as regiões varia aproximadamente de 88% ~ 90% cainda vertiginosamente após esse ano para em média de 60%. Tendo em vista o tamanho do capital as chances de passar do primeiro ano melhoram, mas apenas se for superior a 100k.

Premissa 2. Empresas entre 0 e 3 anos devem ser tratadas como grupo crítico. Empresas entre 3 e 5 anos ainda são relevantes, mas com risco menor

Pelos gráficos, a idade é uma das variáveis mais fortes. A taxa de falência cai conforme a empresa amadurece:

Faixa de idade	Leitura
0–1 ano	risco extremamente alto
1–3 anos	risco muito alto

Faixa de idade	Leitura
3–5 anos	risco alto, mas menor
acima de 5 anos	fora do foco principal da análise

Premissa 3. Região aliado a faixa etária e capital podem determinar a continuidade da empresa

- Sudeste e Sul: Chance de falência de 90% no primeiro ano para micro-empresas sendo São Paulo o estado com melhores possibilidades, sendo necessário alcançar um capital e pelo menos 100K até o 5 ano para definir uma probabilidade maior de continuidade.
- Norte e Nordeste: Chance de falência de 80% no primeiro ano para micro-empresas, sendo necessário alcançar um capital e pelo menos 500k até o 5 ano para definir uma probabilidade maior de continuidade.

Premissa 4. Natureza jurídica

- Ter a natureza jurídica classificadas como Sociedade Simples Limitada, Sociedade Simples Pura ou Sociedade Empresária Limitada para aumentar a chance de não falir dentro de 5 anos.

Premissa 5. Tempos relevantes

Para as empresas não falirem precisam ter:

- Tempo como MEI de até 1 ano e 3 meses;
- E ser da categoria de Opção Simples de 2 a 3 anos;

Premissa 6. CNAE

Caso o CNAE da empresa estiver relacionado as categorias a seguir, o CNPJ estará inserido em um cenário em que a taxa de falência média é 90%:

- Comércio varejista de artigos do vestuário e acessórios

- Lanchonetes, casas de chá, de sucos e similares

Premissa 7. Empresas com capital social até 50k devem ser priorizadas como grupo-alvo de risco. Até 100k ainda merece atenção

O gráfico de faixa de capital mostra uma relação muito clara: quanto menor o capital social, maior a taxa de falência.

Premissa 8. A combinação “empresa jovem + baixo capital” é o principal gatilho de risco

O mapa de calor geral deixa bem claro que a variável mais forte não é só idade ou capital isoladamente, mas a interação entre as duas. O maior risco aparece quando a empresa tem:

- idade entre 0 e 1 ano;
- capital até 100k;
- principalmente capital até 10k.

Também há risco alto em:

- idade entre 1 e 3 anos;
- capital até 100k.

Premissa: criar uma variável combinada do tipo: risco_idade_capital. Assim teremos uma variável única que servirá para o modelo que poderá eliminar duas colunas.

Premissa 9. Porte da empresa deve entrar como variável explicativa forte

Nos seus gráficos, empresas Micro têm taxa de falência maior que empresas Pequenas. Além disso, a literatura do Sebrae também aponta maior mortalidade entre MEIs e microempresas quando comparadas a empresas de pequeno porte.

Premissa 10. Hipótese central da análise

A probabilidade de uma empresa falir nos primeiros 5 anos aumenta principalmente quando ela é jovem, possui baixo capital social, é microempresa, pertence a CNAEs de comércio/serviços com alta mortalidade, possui situação jurídica concentrada em grupos de alto risco e está localizada em UFs/regiões com maior taxa observada. Por outro lado, tempo adequado como MEI e tempo de Opção pelo Simples entre 2 e 3 anos aparecem como possíveis fatores protetivos.